Chapter 6
Memory Organization

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

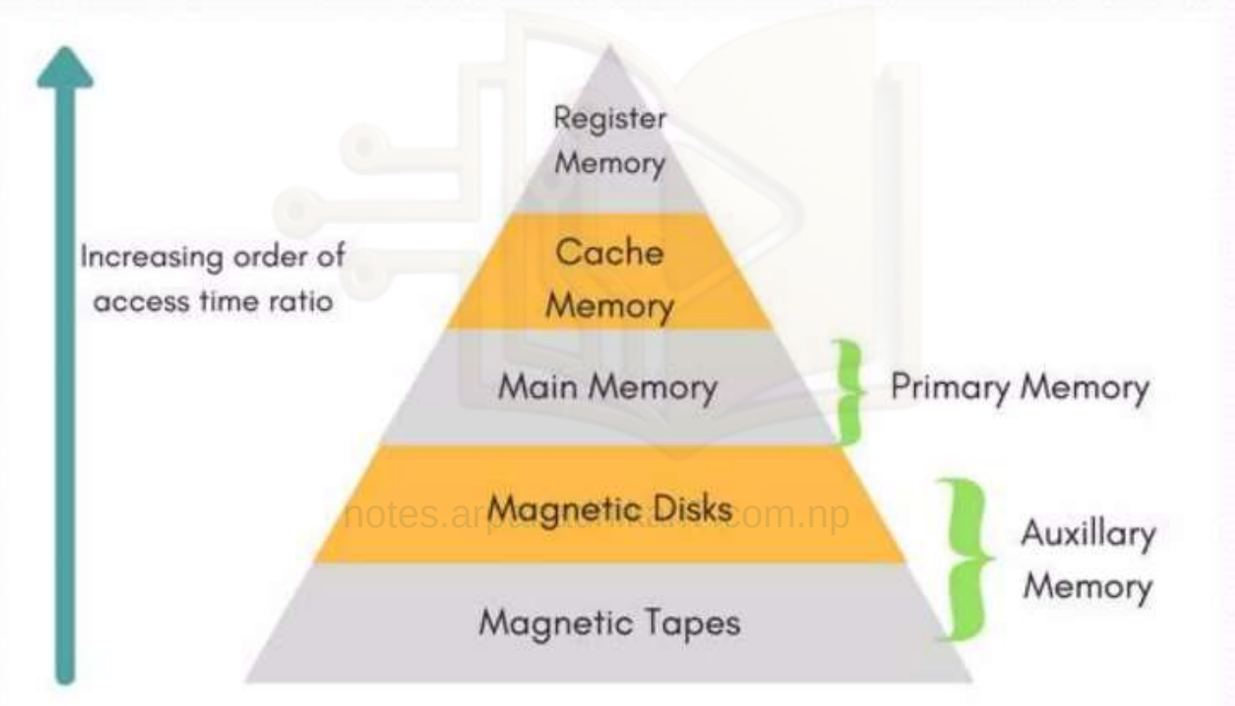
Memory Hierarchy

- Computer Memory Hierarchy is a pyramid structure. A memory is just like a human brain. It is used to store data and instructions
- The memory unit is an essential component in any digital computer since it is needed for storing programs and data.
- The memory unit that directly communicate with CPU is called the main memory .
- Devices that provide backup storage are called auxiliary memory.
- A special very-high-speed memory called cache is used to increase the speed of processing.

notes.arpanadhikari7.com.np

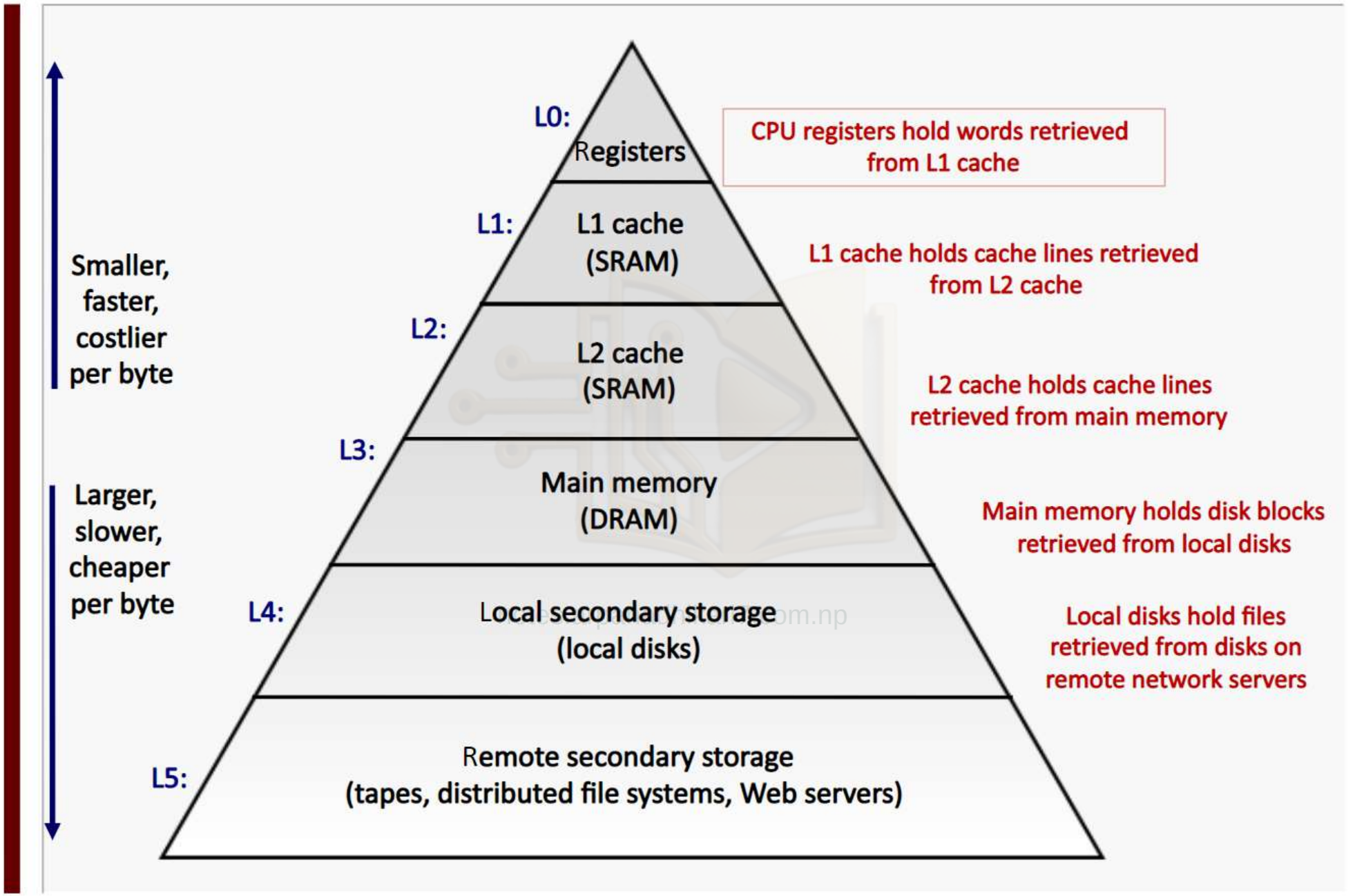
Available on notes.arpanadhikari7.com.np

Diagram of Memory Hierarchy



Available on notes.arpanadhikari7.com.np

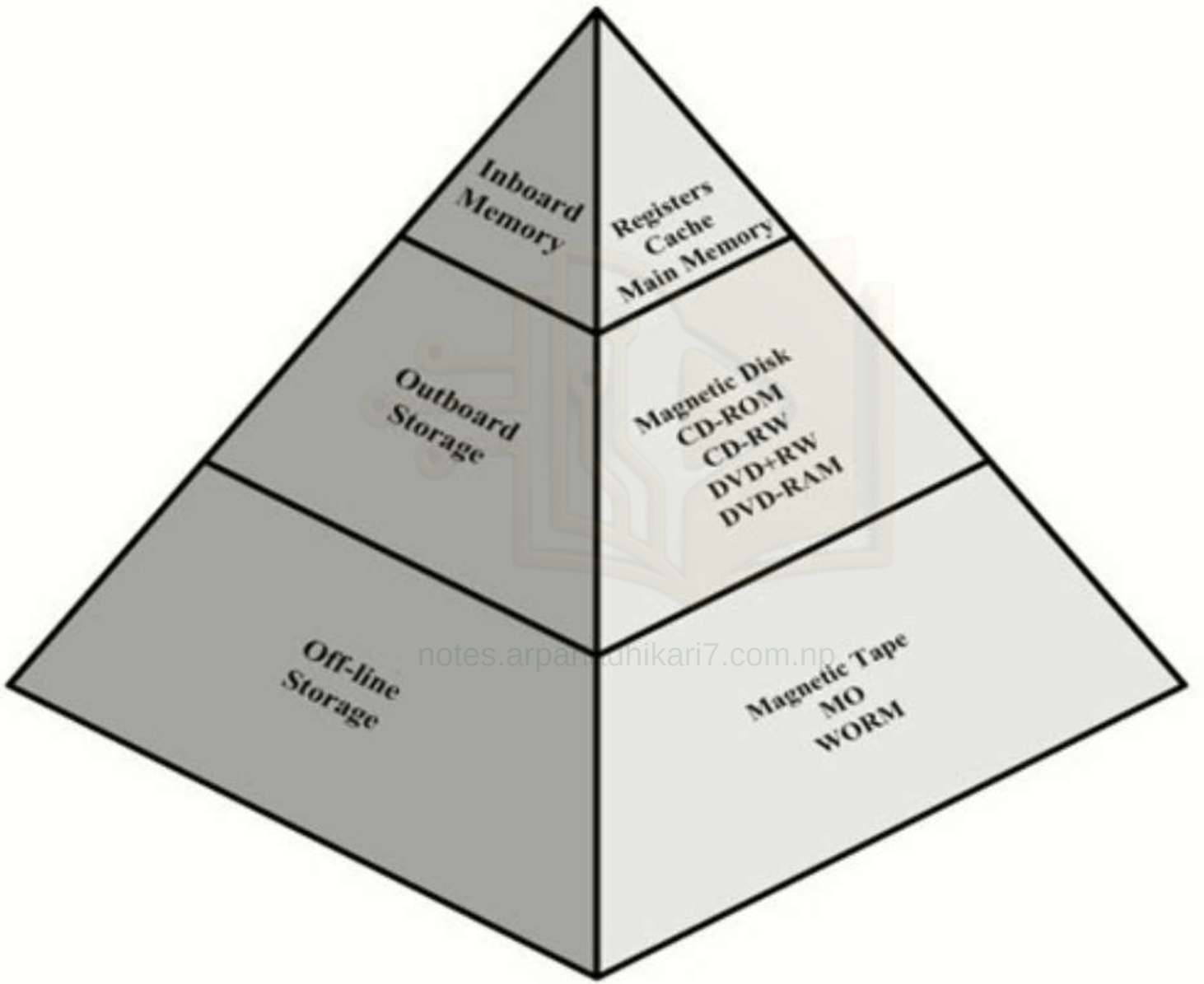
An Example Memory Hierarchy



Available on notes.arpanadhikari7.com.np

The Memory hierarchy

Computer memory system over view



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Memory Characteristics

- Location
- Capacity
- Unit of transfer
- Access method
- Performance
- Physical type
- Physical characteristics
- Organisation

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

1. Location

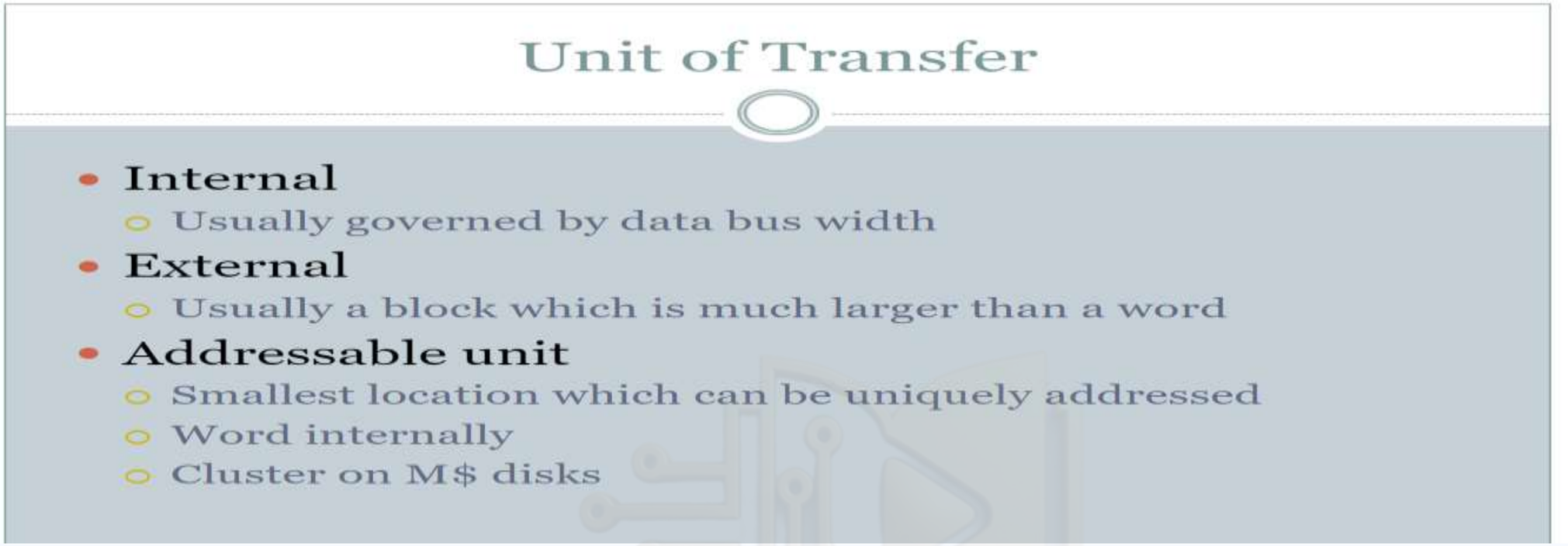
- Refers to whether memory is internal and external.
- Internal memory is often equated with main memory.
- Processor requires its own local memory (registers).
- Cache is another form of internal memory.
- External memory consists of peripheral storage devices that are accessible to the processor via I/O controllers.

notes.arpanadhikari7.com.np

2. Capacity

- Memory is typically expressed in terms of **bytes** or **words.**
- Common word lengths are 8, 16, and 32 bits.

Available on notes.arpanadhikari7.com.np



3. Unit of transfer

- For internal memory the unit of transfer is equal to the number of electrical lines into and out of the memory module.
- For external memory, data are transferred in much larger units than a word (blocks).

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

4. Method of Accessing Units of Data

Sequential access

- Memory is organized into units of data called records
- Access must be made in a specific linear sequence
- Access time is variable
- Ex. Tape

Direct access

- Involves a shared read-write mechanism
- Individual blocks or records have a unique address based on physical location
- Access time is variable
- Ex. Disk

Random access

- Each addressable location in memory has a unique, physically wired-in addressing mechanism
- The time to access a given location is independent of the sequence of prior accesses and is constant
- Any location can be selected at random and directly addressed and accessed
- Ex. Main memory (RAM) and some cache systems

Associative

- A word is retrieved based on a portion of its contents rather than its address
- Each location has its own addressing mechanism and retrieval time is constant independent of location or prior access patterns
- Ex. Cache memories

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

5. Capacity and Performance

- The two most important characteristics of memory.

- Three performance parameters are used:

a. Access time (latency)

- For random-access memory/ it is the time it takes to perform a read or write operation.
- For non-random-access memory/ it is the time it takes to position the read-write mechanism at the desired location.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

b. Memory cycle time

- Access time plus any additional time required before second access can commence
- Additional time may be required for transients to die out on signal lines or to regenerate data if they are read destructively.
- Concerned with the system bus, not the processor.

c. Transfer rate

- The rate at which data can be transferred into or out of a memory unit.
- For random-access memory it is equal to 1/(cycle time).

Available on notes.arpanadhikari7.com.np

Performance

- **Transfer rate**
  - Rate which data transferred into or out of memory unit (for random-access memory)
  - For non-random-access memory

$$T_N = T_A + \frac{N}{R}$$

For random access memory it is 1/(cycle time).

T_N = Average time to read or write N bits

T_A = Average access time

N = Number of bits

R = Transfer rate, in bits per second(bps)

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

6. Physical type of memory

The most common forms are:

- Semiconductor memory (RAM).
- Magnetic surface memory (disk and tape).
- Optical (CD & DVD).
- Magneto-optical.

7. Physical characteristics of data storage:

Volatility

notes.arpanadhikari7.com.np

- Volatile memory: Information decays naturally or is lost when electrical power is switched off.
- Nonvolatile memory: Once recorded, information remains without damage until deliberately changed.

Available on notes.arpanadhikari7.com.np

- No electrical power is needed to retain information.
- Magnetic-surface memories are nonvolatile.
- Semiconductor memory may be either volatile or nonvolatile.

Erasable

- Non-erasable memory Cannot be altered, except by destroying the storage unit.
- Semiconductor memory of this type is known as read-only memory (ROM).

notes.arpanadhikari7.com.np

8. Organization

"the physical arrangement of bits to form words".

For RAM, the organization is a key design issue.

Available on notes.arpanadhikari7.com.np

The Bottom Line

- How much?
  - Capacity
- How fast?
  - Time is money
- How expensive?

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Table 5.1 Semiconductor Memory Types

Memory Type	Category	Erasure	Write Mechanism	Volatility
Random-access memory (RAM)	Read-write memory	Electrically, byte-level	Electrically	Volatile
Read-only memory (ROM)	Read-only memory	Not possible	Masks	Nonvolatile
Programmable ROM (PROM)			Electrically	
Erasable PROM (EPROM)	Read-mostly memory	UV light, chip-level		
Electrically Erasable PROM (EEPROM)		Electrically, byte-level		
Flash memory		Electrically, block-level		

Available on notes.arpanadhikari7.com.np

RAM

- RAM is stands for **Random Access Memory**.
- RAM is the internal memory of the CPU for storing data, program, and program result.
- It is a read/write memory which stores data until the machine is working.
- RAM can be accessed randomly but it is very expensive.
- RAM is volatile, i.e. data stored in it is lost when we switch off the computer.
- Types of RAM - DRAM, SRAM.



RAM

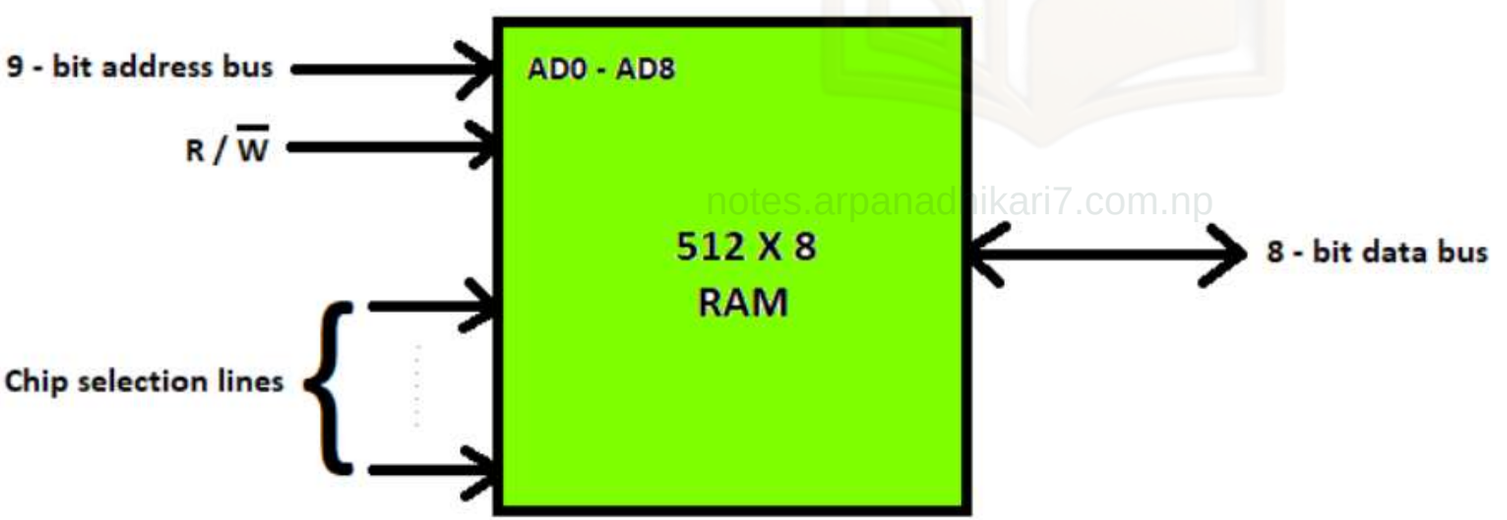
Available on notes.arpanadhikari7.com.np

RAM(Random Access Memory) is a part of computer's Main Memory which is directly accessible by CPU. RAM is used to Read and Write data into it which is accessed by CPU randomly. RAM is volatile in nature, it means if the power goes off, the stored information is lost. RAM is used to store the data that is currently processed by the CPU. Most of the programs and data that are modifiable are stored in RAM.

Integrated RAM chips are available in two form:

1. SRAM(Static RAM)
2. DRAM(Dynamic RAM)

The block diagram of RAM chip is given below.

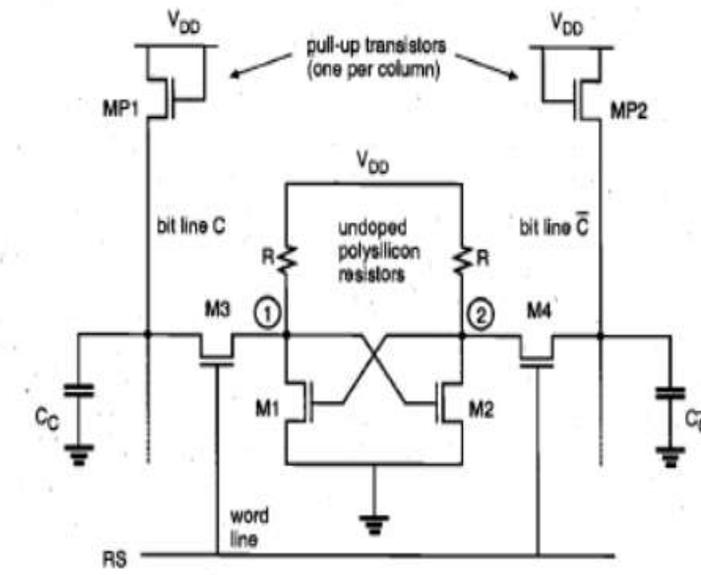


notes.arpanadhikari7.com.np

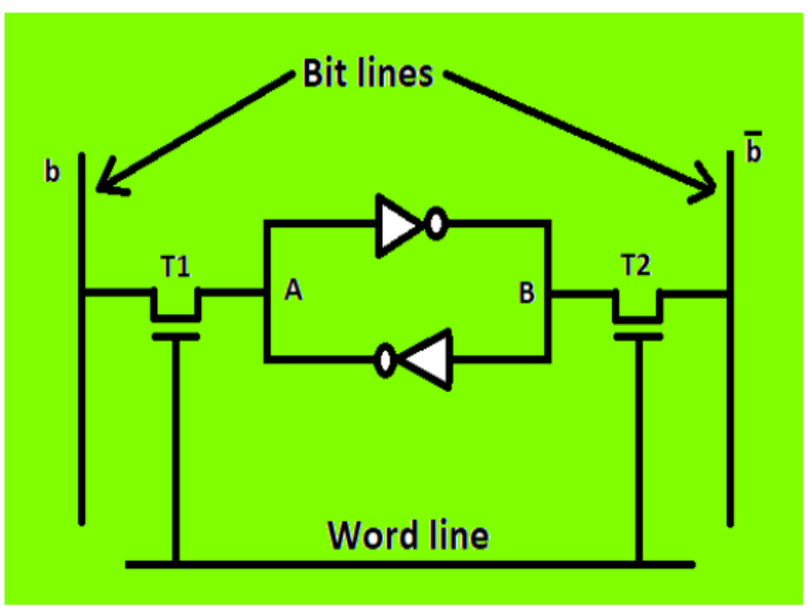
Available on notes.arpanadhikari7.com.np

1. SRAM :

The SRAM memories consist of circuits capable of retaining the stored information as long as the power is applied. That means this type of memory requires constant power. SRAM memories are used to build Cache Memory.



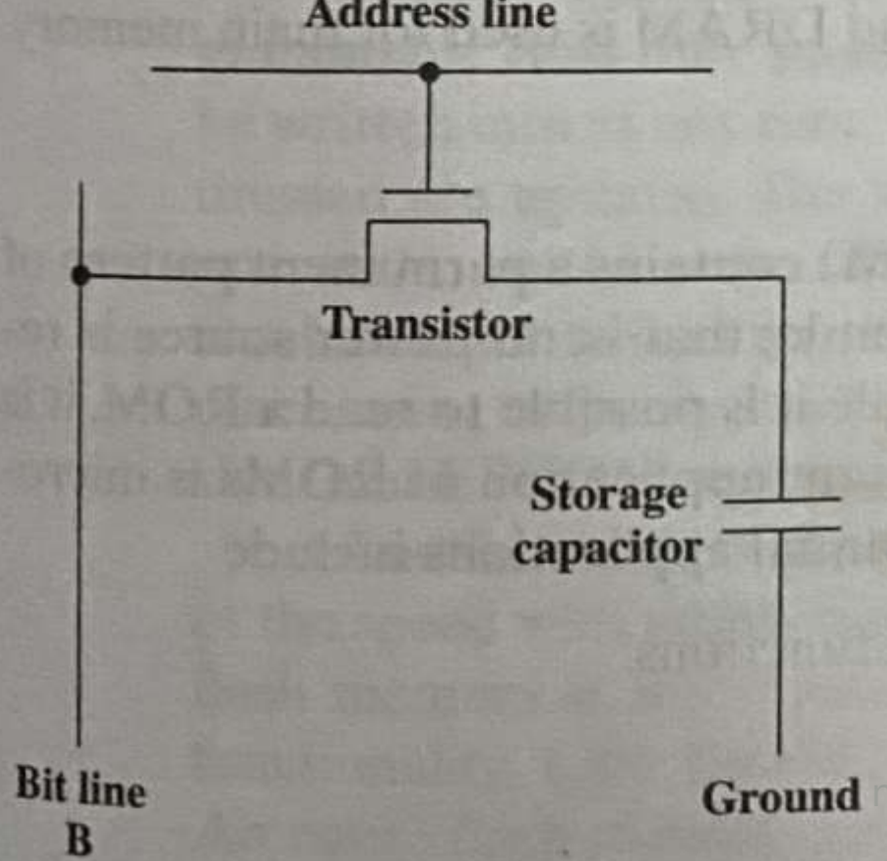
SRAM Memory Cell: Static memories(SRAM) are memories that consist of circuits capable of retaining their state as long as power is on. Thus this type of memory is called volatile memory. The below figure shows a cell diagram of SRAM. A latch is formed by two inverters connected as shown in the figure. Two transistors T1 and T2 are used for connecting the latch with two-bit lines. The purpose of these transistors is to act as switches that can be opened or closed under the control of the word line, which is controlled by the address decoder. When the word line is at 0-level, the transistors are turned off and the latch remains its information. For example, the cell is at state 1 if the logic value at point A is 1 and at point, B is 0. This state is retained as long as the word line is not activated.



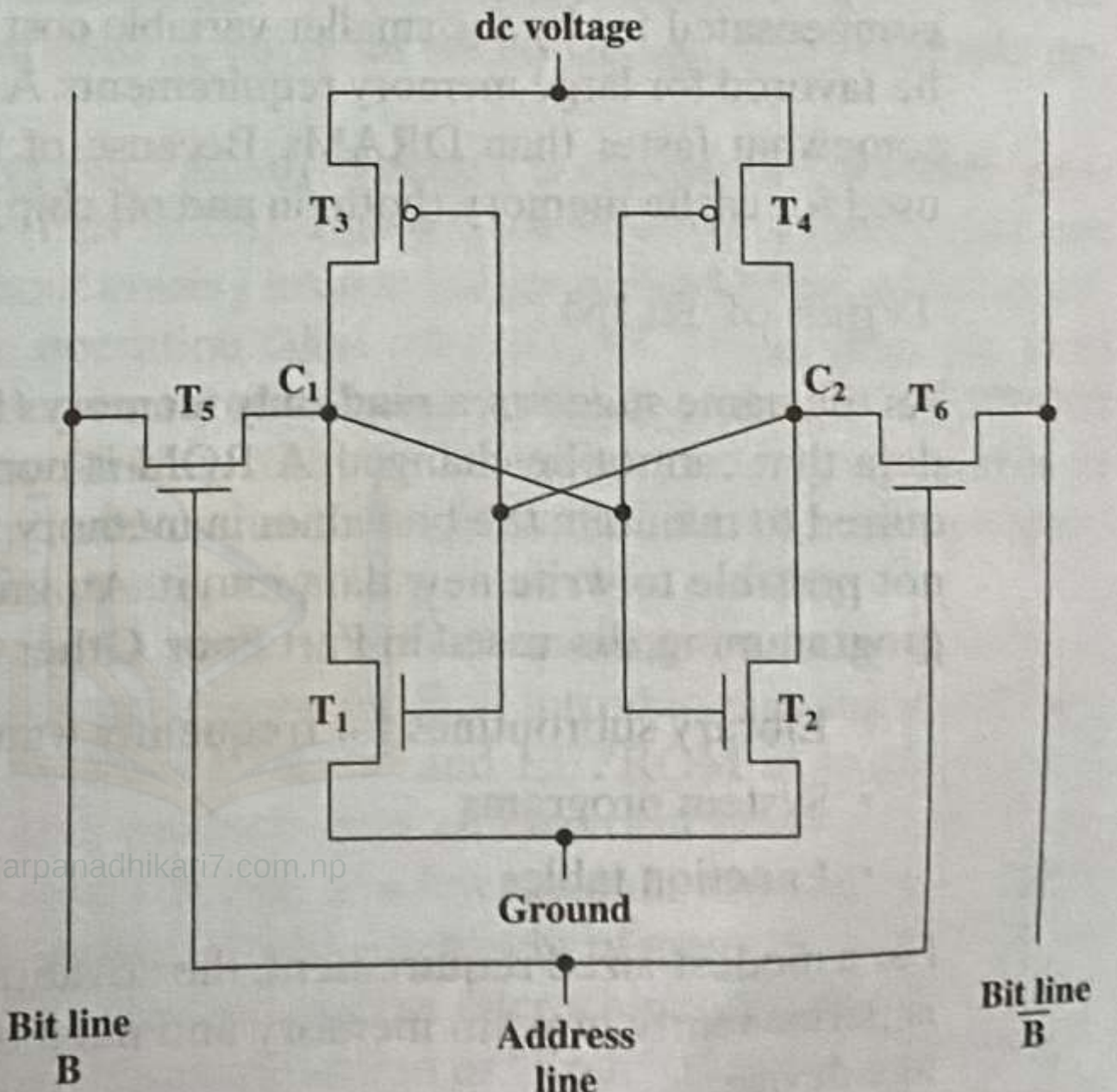
For **Read operation**, the word line is activated by the address input to the address decoder. The activated word line closes both the transistors (switches) T1 and T2. Then the bit values at points A and B can transmit to their respective bit lines. The sense/write circuit at the end of the bit lines sends the output to the processor.

For **Write operation**, the address provided to the decoder activates the word line to close both the switches. Then the bit value that is to be written into the cell is provided through the sense/write circuit and the signals in bit lines are then stored in the cell.

Available on notes.arpanadhikari7.com.np



(a) Dynamic RAM (DRAM) cell



(b) Static RAM (SRAM) cell

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

2. DRAM :

DRAM stores the binary information in the form of electric charges applied to capacitors. The stored information on the capacitors tends to lose over a period of time and thus the capacitors must be periodically recharged to retain their usage. The main memory is generally made up of DRAM chips.

DRAM Memory Cell: Though SRAM is very fast, but it is expensive because of its every cell requires several transistors. Relatively less expensive RAM is DRAM, due to the use of one transistor and one capacitor in each cell, as shown in the below figure., where C is the capacitor and T is the transistor. Information is stored in a DRAM cell in the form of a charge on a capacitor and this charge needs to be periodically recharged.

For storing information in this cell, transistor T is turned on and an appropriate voltage is applied to the bit line. This causes a known amount of charge to be stored in the capacitor. After the transistor is turned off, due to the property of the capacitor, it starts to discharge. Hence, the information stored in the cell can be read correctly only if it is read before the charge on the capacitors drops below some threshold value.

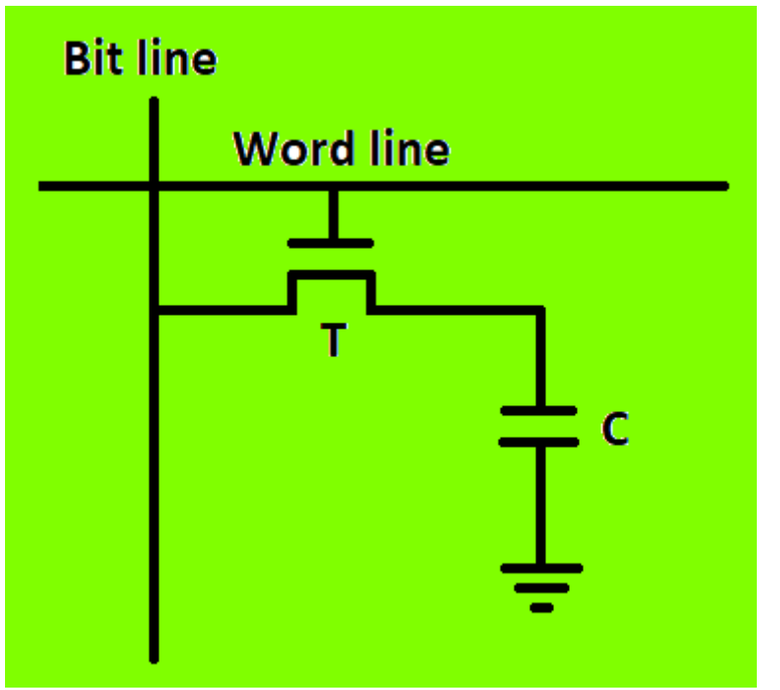


Figure: Single DRAM cell

Available on notes.arpanadhikari7.com.np

Difference between SRAM and DRAM :

Below table lists some of the differences between SRAM and DRAM:

SRAM	DRAM
1. SRAM has lower access time, so it is faster compared to DRAM.	1. DRAM has higher access time, so it is slower than SRAM.
2. SRAM is costlier than DRAM.	2. DRAM costs less compared to SRAM.
3. SRAM requires constant power supply, which means this type of memory consumes more power.	3. DRAM offers reduced power consumption, due to the fact that the information is stored in the capacitor.
4. Due to complex internal circuitry, less storage capacity is available compared to the same physical size of DRAM memory chip.	4. Due to the small internal circuitry in the one-bit memory cell of DRAM, the large storage capacity is available.
5. SRAM has low packaging density.	5. DRAM has high packaging density.

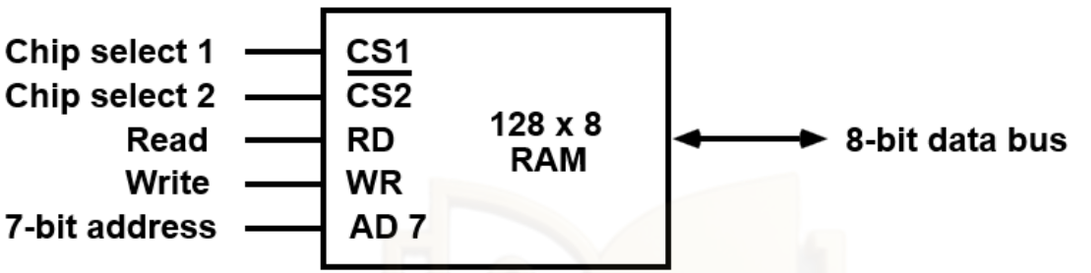
notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

MAIN MEMORY

RAM and ROM Chips

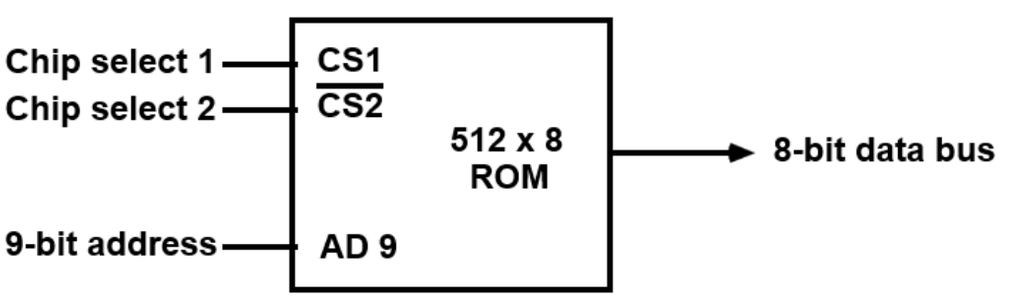
Typical RAM chip



CS1	$\overline{CS2}$	RD	WR	Memory function	State of data bus
0	0	x	x	Inhibit	High-impedence
0	1	x	x	Inhibit	High-impedence
1	0	0	0	Inhibit	High-impedence
1	0	0	1	Write	Input data to RAM
1	0	1	x	Read	Output data from RAM
1	1	x	x	Inhibit	High-impedence

notes.arpanadhikari7.com.np

Typical ROM chip



Available on notes.arpanadhikari7.com.np

MEMORY ADDRESS MAP

Address space assignment to each memory chip

Example: 512 bytes RAM and 512 bytes ROM

Component	Hexa address	Address bus									
		10	9	8	7	6	5	4	3	2	1
RAM 1	0000 - 007F	0	0	0	x	x	x	x	x	x	x
RAM 2	0080 - 00FF	0	0	1	x	x	x	x	x	x	x
RAM 3	0100 - 017F	0	1	0	x	x	x	x	x	x	x
RAM 4	0180 - 01FF	0	1	1	x	x	x	x	x	x	x
ROM	0200 - 03FF	1	x	x	x	x	x	x	x	x	x

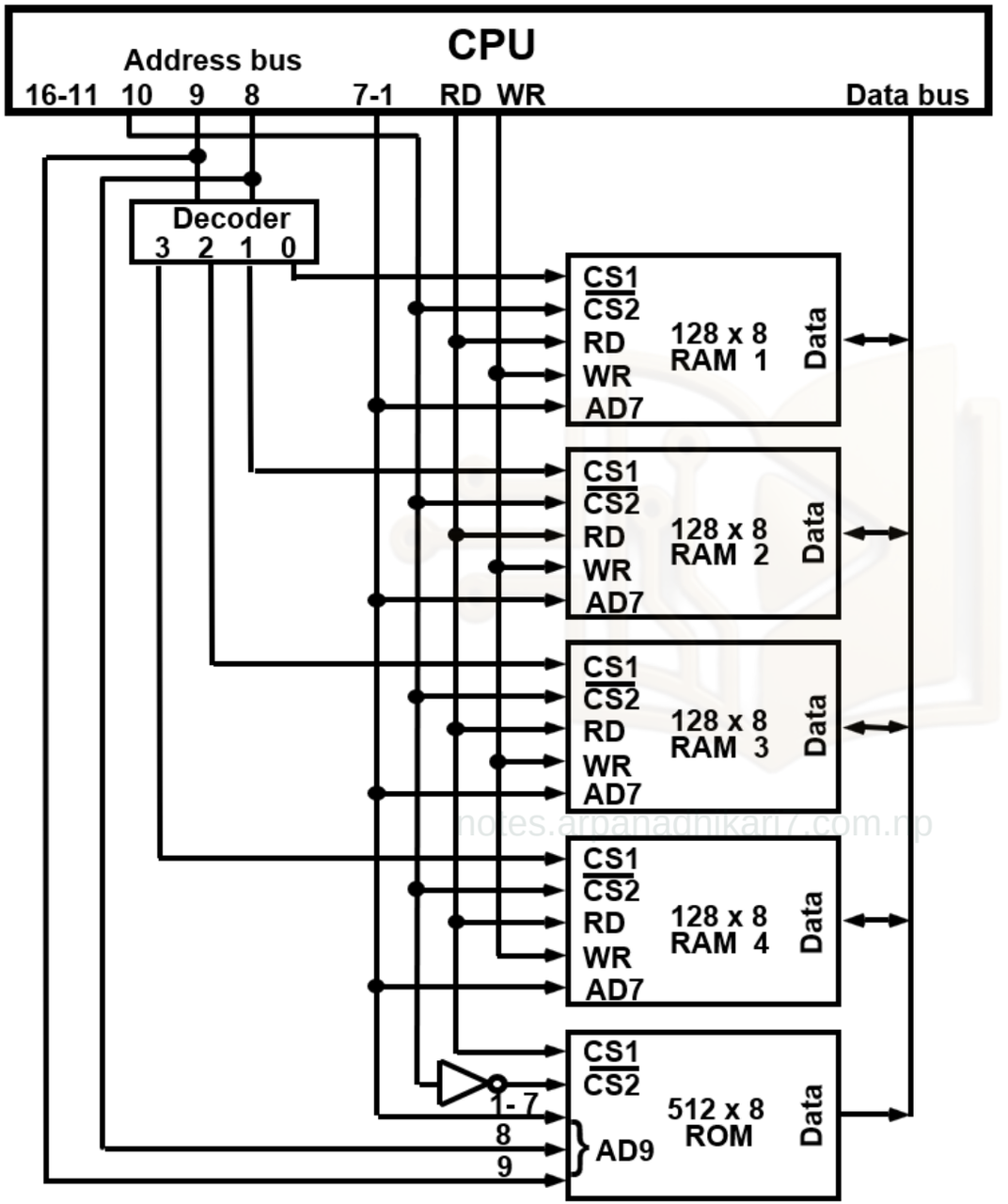
Memory Connection to CPU

- RAM and ROM chips are connected to a CPU through the data and address buses

- The low-order lines in the address bus select the byte within the chips and other lines in the address bus select a particular chip through its chip select inputs

Available on notes.arpanadhikari7.com.np

CONNECTION OF MEMORY TO CPU



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

ROM

- ROM stands for **Read Only Memory**.
- we can only read but cannot write on it.
- It is non-volatile memory.
- Data in ROM can not be modified.
- ROM is comparatively smaller than RAM
- Types of ROM are PROM, EPROM, EEPROM.

notes.arpanadhikari7.com.np



ROM
Read only Memory

Available on notes.arpanadhikari7.com.np

RAID

What is RAID?

- Redundant Array of Inexpensive Disks.

Basic idea is to connect multiple disks together to provide

- Large storage capacity
- Faster access to reading data
- Redundant data

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Why RAID?

- RAID system provides data redundancy, fault tolerance, increased capacity and increased performance.

- RAID has for a long time been something that you only find in large server systems, but lately cheaper **RAID controller card** have made it possible to get a RAID system even for small servers and home computers.

How does RAID works?

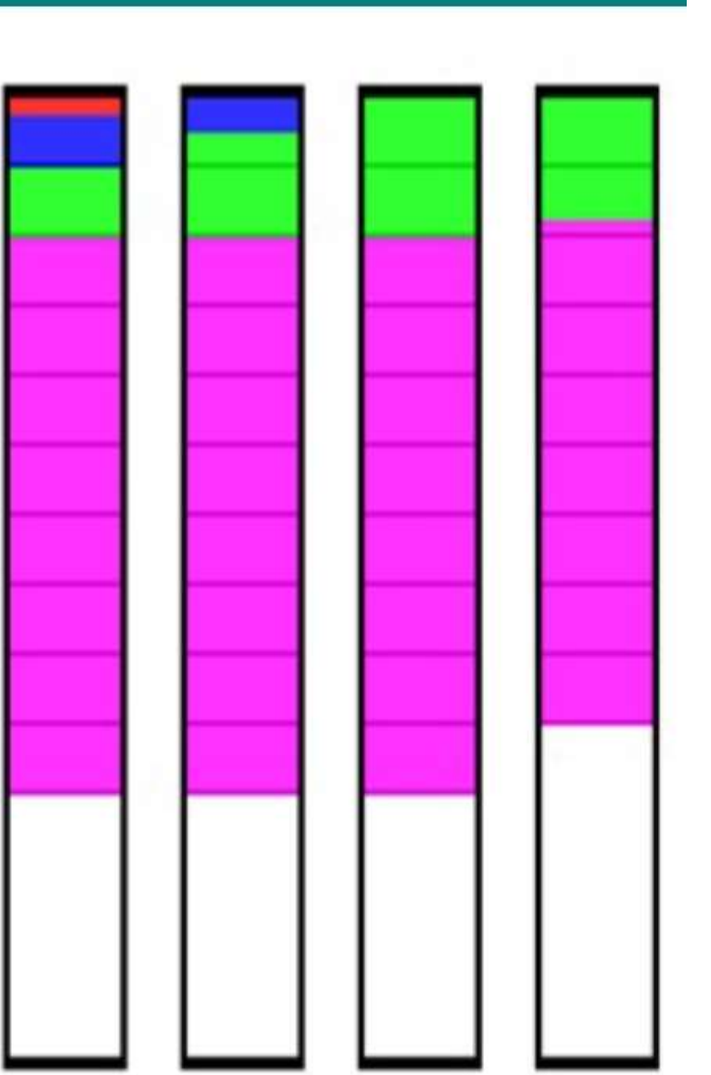
- Take some inexpensive disks and group them together, which will make the system see them as one single disk.

- This is done by using a **RAID controller card** that handle all I/O to the disks, and which knows where the stored data can be found.

Available on notes.arpanadhikari7.com.np

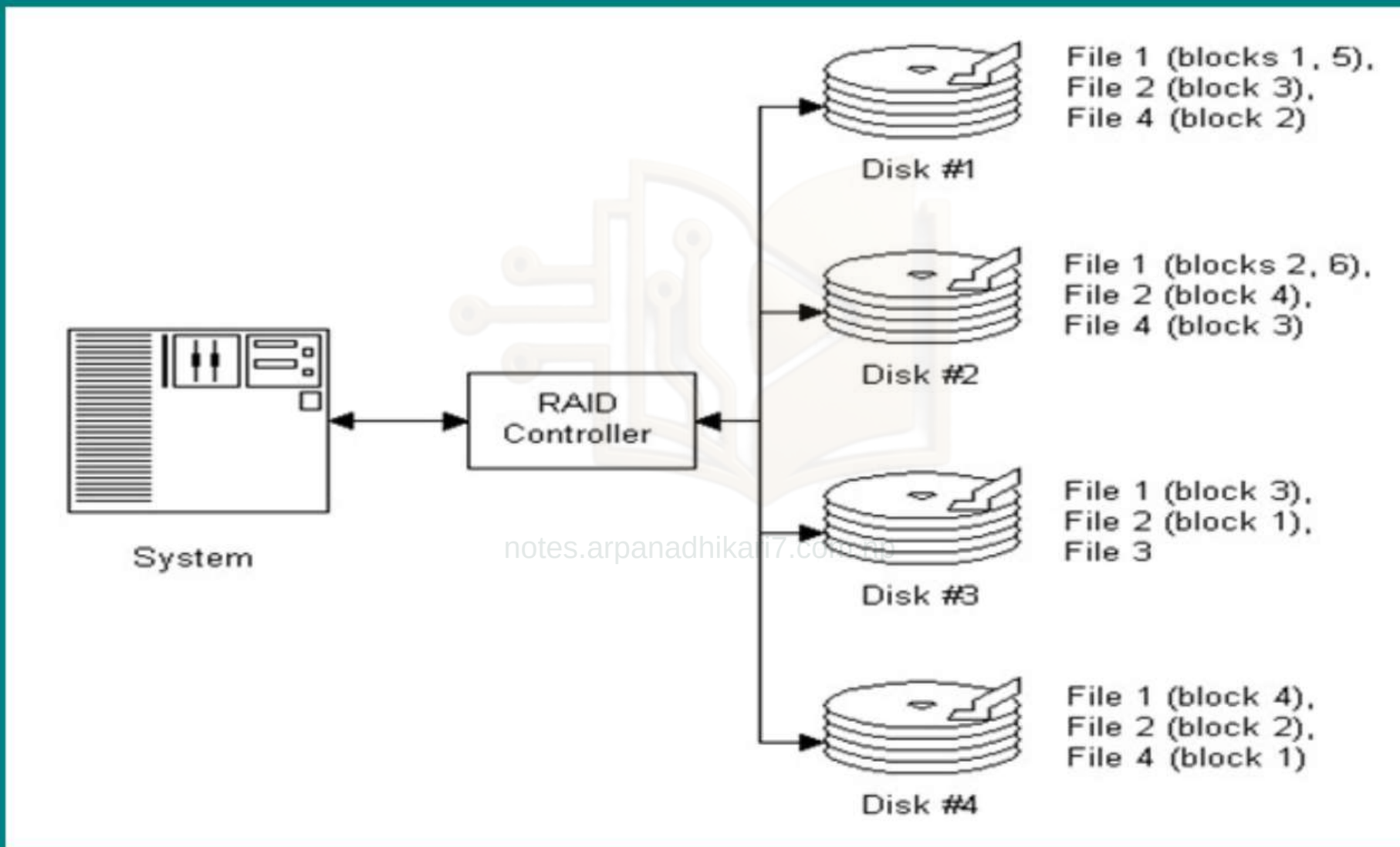
Let's Talk about Striping

- RAID arrays that use striping improve performance by splitting up files into small pieces and distributing them to multiple hard disks.
- File Red 140KB
- File Blue 600KB
- File Green 3.2MB
- File Purple 21.2MB



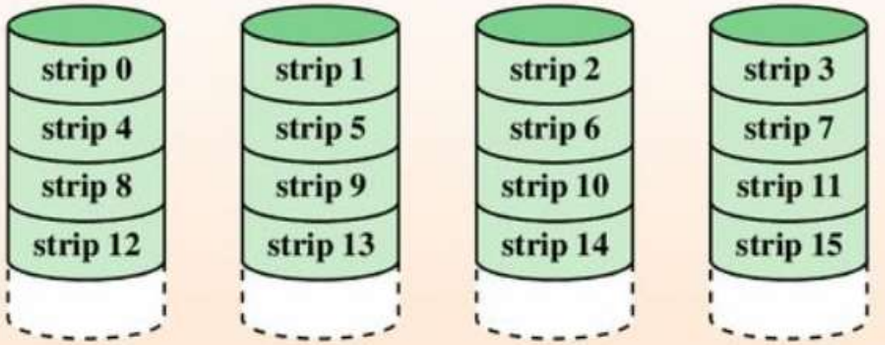
Available on notes.arpanadhikari7.com.np

Block Diagram of a Raid Striping Configuration

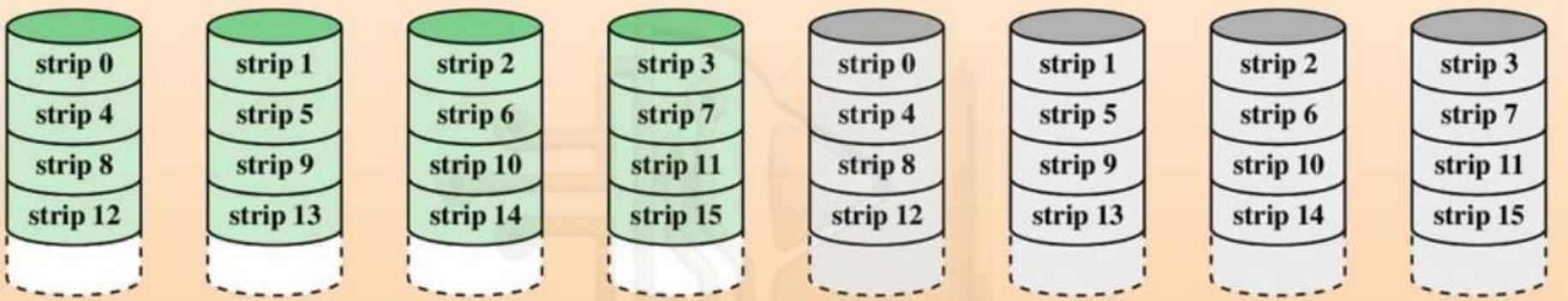


notes.arpanadhikari7.com.np

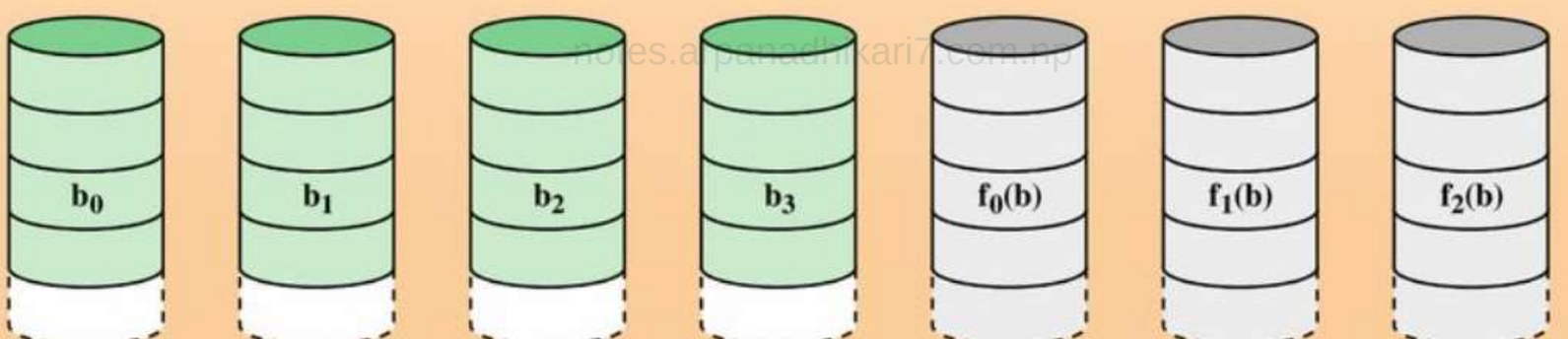
Available on notes.arpanadhikari7.com.np



(a) RAID 0 (non-redundant)

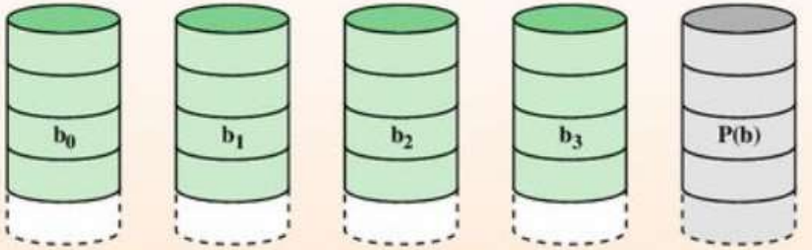


(b) RAID 1 (mirrored)

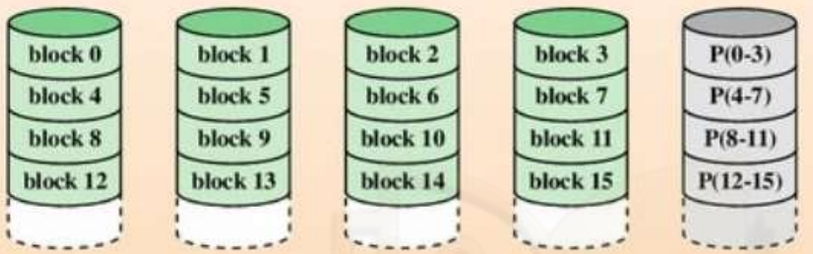


(c) RAID 2 (redundancy through Hamming code)

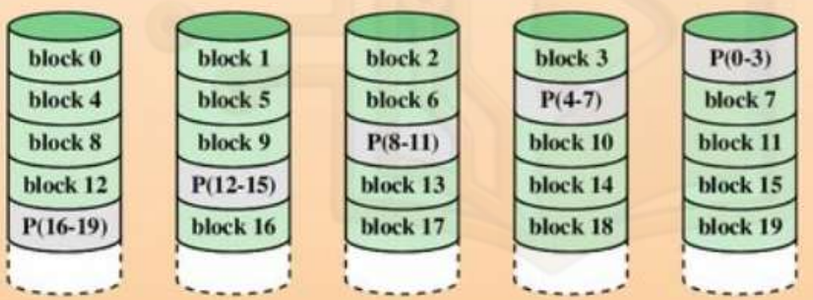
Available on notes.arpanadhikari7.com.np



(d) RAID 3 (bit-interleaved parity)

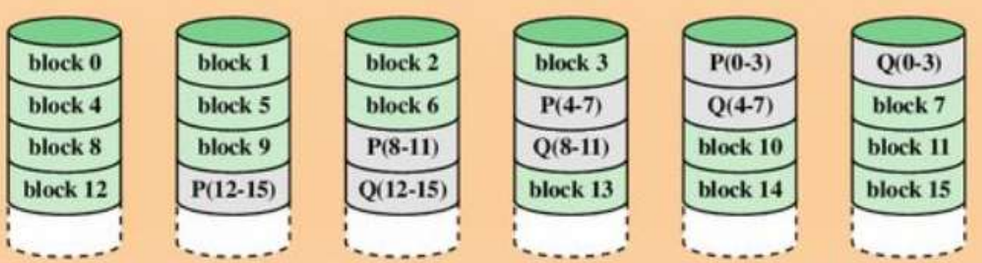


(e) RAID 4 (block-level parity)



notes.arpanadhikari7.com.np

(f) RAID 5 (block-level distributed parity)



(g) RAID 6 (dual redundancy)

Available on notes.arpanadhikari7.com.np

RAID 0

- No redundancy
- Data striped across all disks
- Round Robin striping
- Increase speed
  - Multiple data requests probably not on same disk
  - Disks seek in parallel
  - A set of data is likely to be striped across multiple disks

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

RAID 1

- Mirrored Disks
- Data is striped across disks
- 2 copies of each stripe on separate disks
- Read from either
- Write to both
- Recovery is simple
  - —Swap faulty disk & re-mirror
  - —No down time
- Expensive

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

RAID 2

- Disks are synchronized
- Very small stripes
  - —Often single byte/word
- Error correction calculated across corresponding bits on disks
- Multiple parity disks store Hamming code error correction in corresponding positions
- Lots of redundancy
  - —Expensive
  - —Not used

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

RAID 3

- Similar to RAID 2
- Only one redundant disk, no matter how large the array
- Simple parity bit for each set of corresponding bits
- Data on failed drive can be reconstructed from surviving data and parity info
- Very high transfer rates

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

RAID 4

- Each disk operates independently
- Good for high I/O request rate
- Large stripes
- Bit by bit parity calculated across stripes on each disk
- Parity stored on parity disk

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

RAID 5

- Like RAID 4
- Parity striped across all disks
- Round robin allocation for parity stripe
- Avoids RAID 4 bottleneck at parity disk
- Commonly used in network servers

RAID 6

- Two parity calculations
- Stored in separate blocks on different disks
- User requirement of N disks needs N+2
- High data availability
  - —Three disks need to fail for data loss
  - —Significant write penalty

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Table 6.3
RAID Levels

Category	Level	Description	Disks Required	Data Availability	Large I/O Data Transfer Capacity	Small I/O Request Rate
Striping	0	Nonredundant	N	Lower than single disk	Very high	Very high for both read and write
Mirroring	1	Mirrored	$2N$	Higher than RAID 2, 3, 4, or 5; lower than RAID 6	Higher than single disk for read; similar to single disk for write	Up to twice that of a single disk for read; similar to single disk for write
Parallel access	2	Redundant via Hamming code	$N+m$	Much higher than single disk; comparable to RAID 3, 4, or 5	Highest of all listed alternatives	Approximately twice that of a single disk
	3	Bit-interleaved parity	$N+1$	Much higher than single disk; comparable to RAID 2, 4, or 5	Highest of all listed alternatives	Approximately twice that of a single disk
Independent access	4	Block-interleaved parity	$N+1$	Much higher than single disk; comparable to RAID 2, 3, or 5	Similar to RAID 0 for read; significantly lower than single disk for write	Similar to RAID 0 for read; significantly lower than single disk for write
	5	Block-interleaved distributed parity	$N+1$	Much higher than single disk; comparable to RAID 2, 3, or 4	Similar to RAID 0 for read; lower than single disk for write	Similar to RAID 0 for read; generally lower than single disk for write
	6	Block-interleaved dual distributed parity	$N+2$	Highest of all listed alternatives	Similar to RAID 0 for read; lower than RAID 5 for write	Similar to RAID 0 for read; significantly lower than RAID 5 for write

N = number of data disks; m proportional to log N

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Associative Memory

- Many data processing applications require searching of an item efficiently.
- An Assembler program searches the symbol address table in order to extract the symbol's binary equivalent.
- The number of accesses to memory depends on the location of the item and the efficiency of search algorithm.
- Many Search algorithm have been developed to minimize the number of access while searching for an item in a random or sequential access memory.
- The time required to find an item stored in memory can be reduced considerably if stored data can be identified by the content of the data itself rather than an address.
- A memory unit accessed by content is called an associative memory or Content Addressable Memory (CAM).
- This type of memory is accessed simultaneously and in parallel on the basis of data content renther that by specific address or location.
- When a word is written to associative memory no address is given. The memory is capable of finding an empty unused location to store the word.
- When a word is to be read from an associative memory, the content of the word, or part of the word is specified. The memory locates all words which match the specified content and marks them for reading.
- An associative memory is more expensive than a random access memory, so are used in application where search time is very critical and must be very short.

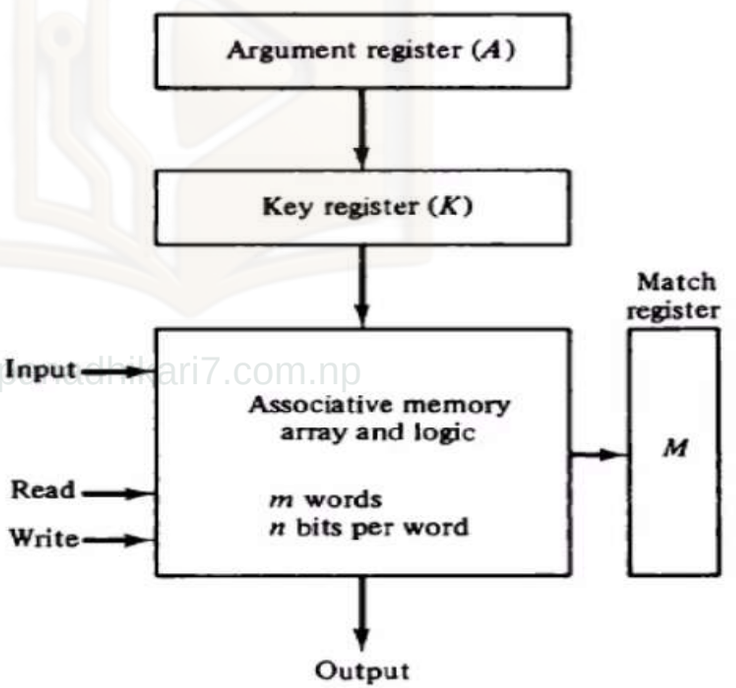
notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Hardware Organization

- It consist of a memory array and logic for m Words with n bits per word.
- Argument register A and key register K Each have n bits.
- The match register M has m bits, one for each memory word.
- Each word in the memory is compared in parallel with the content of the argument register. The words that match the bits of the argument register set a corresponding bit in the match register. Then reading is accomplished by a sequential access to memory

'A' contents word to be searched.
(1 bit for each word).
Key register provides a mask for choosing particular key in augment word, specify which part of augment words needs to be compared with words in memory.
If all bits of key register is 1's the entire words should be compared else the bits having 1's is compared.
Match register m bits for corresponding n number of words.



M has m bits, 1 bit corresponding to each word in the memory array. After the matching process the bits corresponding to matching word in match register is set to 1.

Available on notes.arpanadhikari7.com.np

Key register

- It provides a mask for choosing a particular field or key in the argument word.
- The entire argument is compared with each memory word if the key register contains all 1's. Otherwise, only those bits in the argument that have 1's in their corresponding position of the key register are compared.

Let take an example

A Register 101 111100
K Register 111 000000

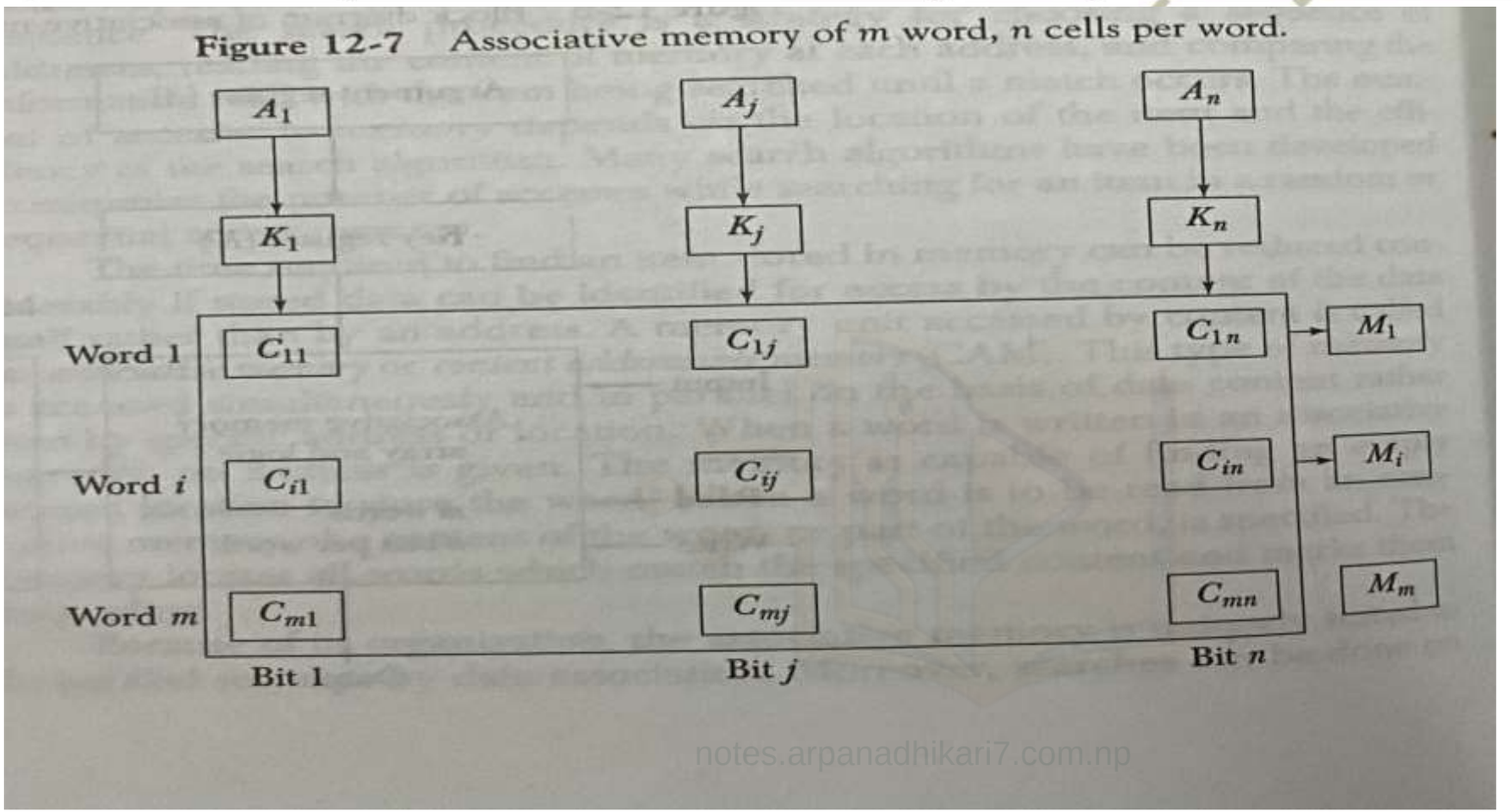
Note: Searching sequential , Matching parallelly.

notes.arpanadhikari7.com.np

Word 1 100 111100 M = 0
Word 2 101 000011 M = 1

Available on notes.arpanadhikari7.com.np

Relationship between the memory array and external registers



Each word is n no. of cells (Cij). Where i= word number and j= bit position)

- Cell C_{ij} is the cell for bit j in word i. A bit A_j in the argument register is compared with all bits in column j of the array provided that $k_j = 1$.
- This is done for all comumns j=1,2,...,n. If match occurs between all the unmasked bits of the argument and the bits in word I, then set M_i to 1. Otherwise set M_i to 0.

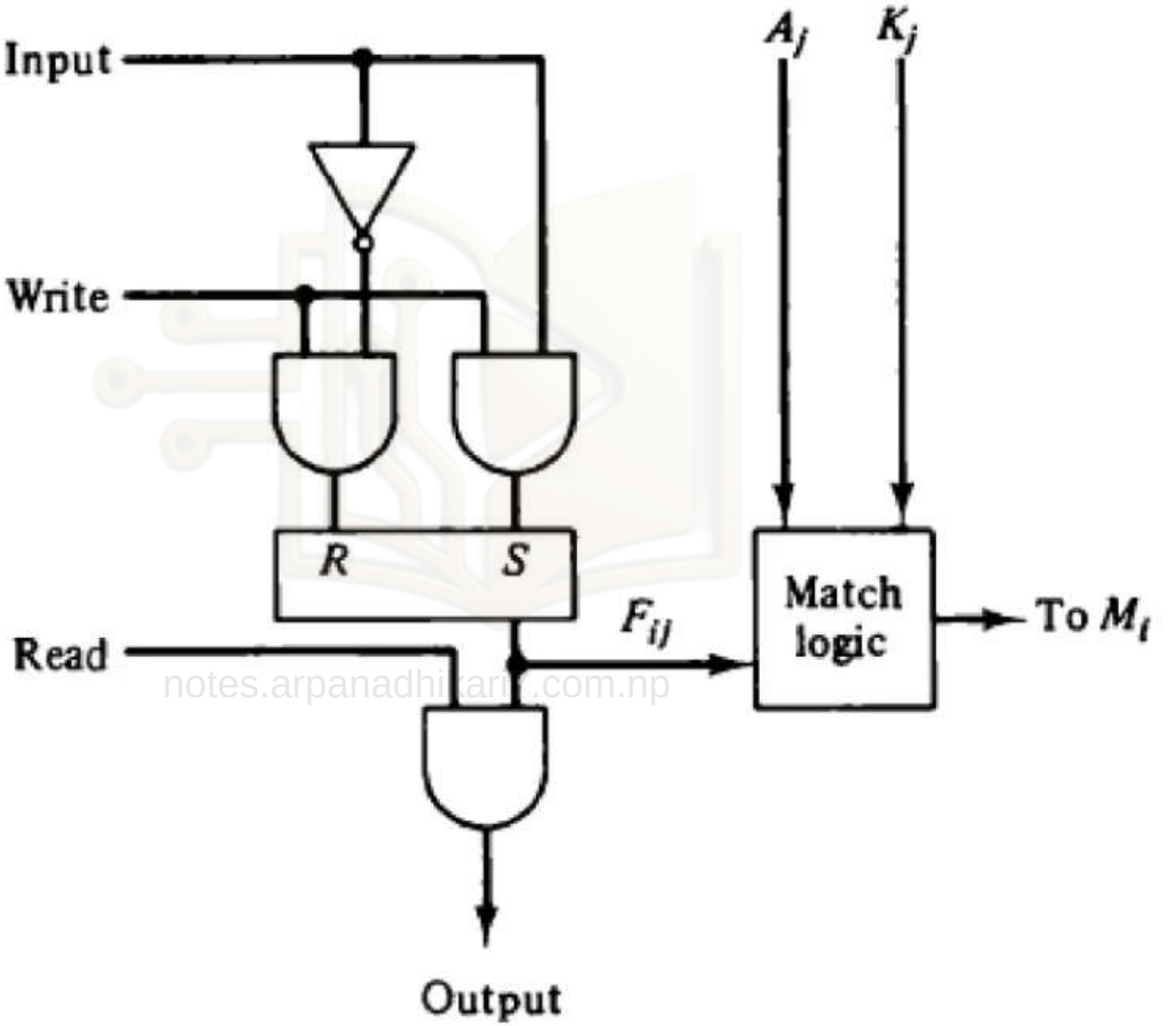
notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

One cell of associative memory

C_{ij}

- It consist of a flip-flop storage element F_{ij} and circuit for reading, writing , and matching the cell.



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Match Logic

- Word i is equal to the argument in A if $A_j = F_{ij}$ for j=1,2,..,n. Two bits are equal if they are both 1 or both 0. The equality of two bits can be expressed logically by Boolean function.

$$x_j = A_j F_{ij} + A_j' F_{ij}'$$

- where $x_j = 1$ if the pair of bits in position j are equal; otherwise, $x_j = 0$.
- For a word i to be equal to the argument in A we must have all x_j variables equal to 1. This is the condition for setting the corresponding match bit M_i to 1. Boolean function for this condition is

$$M_i = x_1 x_2 \ldots x_n$$

- If $k_j = 0$, the corresponding bits of A_j and F_{ij} need no comparison. Only when $K_j = 1$ must they be compared. This requirement is achieved by ORing each term with K_j', thus

$$x_j + K_j' = \begin{cases} x_j, & k_j = 1 \\ 1, & k_j = 0 \end{cases}$$

- The match logic for word i in an associative memory can now be expressed by the following Boolean function.

$$M_i = (x_1 + K_1')(x_2 + K_2')(x_3 + K_3') \ldots (x_n + K_n')$$

- If we substitute the original definition of x_j.

$$M_i = \prod_{j=1}^{n} (A_j F_{ij} + A_j' F_{ij}' + K_j')$$

➔ where $\prod$ is a product symbol designating the AND operation of all n terms.

Available on notes.arpanadhikari7.com.np

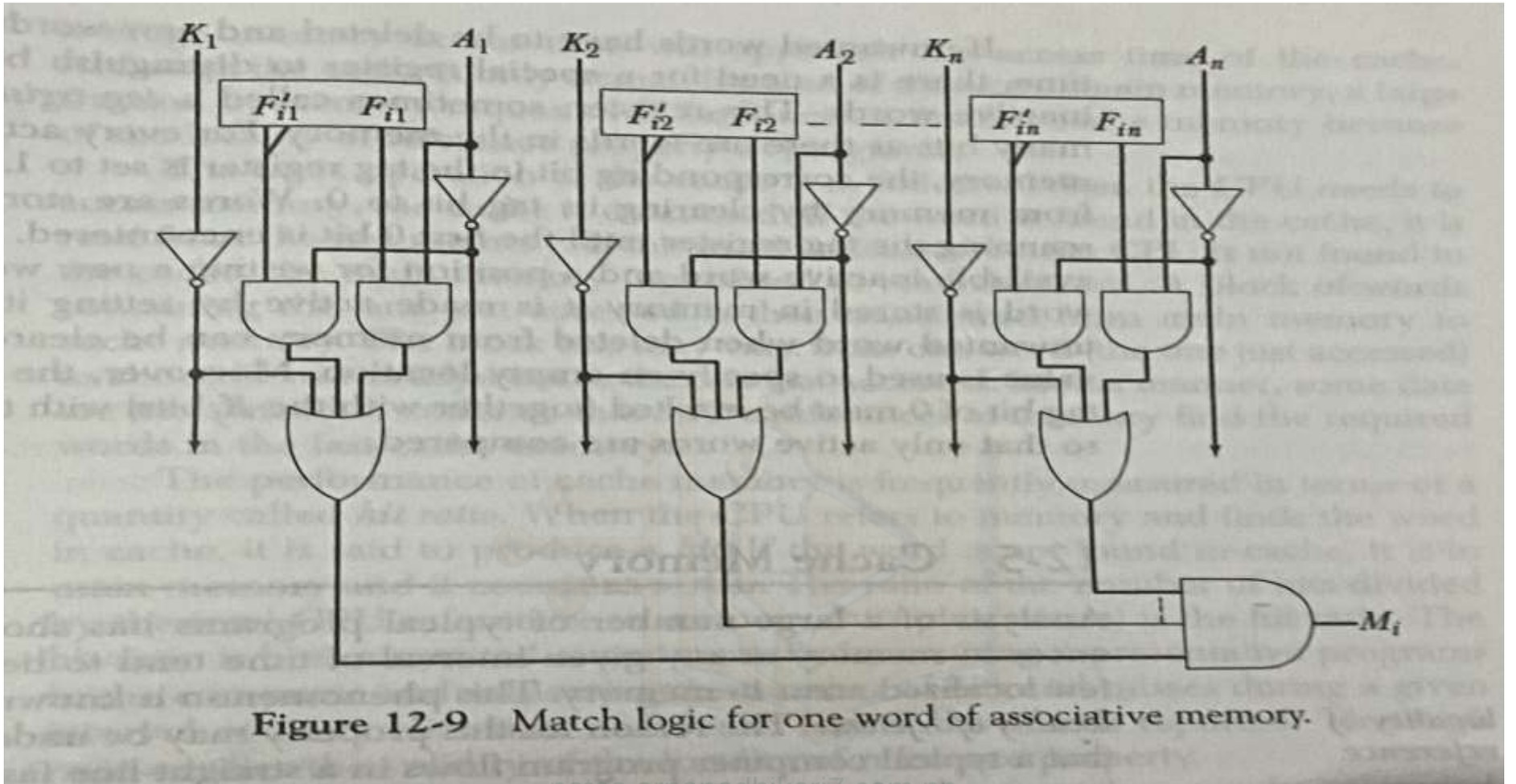


Figure 12-9 Match logic for one word of associative memory.

notes.arpanadhikari7.com.np

- Each cell requires 2 AND gets and one OR gate. The inverters for Aj and K_j are needed once for each column and are used for all bits in the column. The output to all OR gates in the cells of the same word go to the input of the common AND get to Generate the match signal for M_i.

Available on notes.arpanadhikari7.com.np

Read Operation

- if more than one word in memory matches, then read in sequence by applying read signal to each word line with M_i =1.
- If only one word may match, connect output M_i directly to the read line in the same word position.

Write Operation

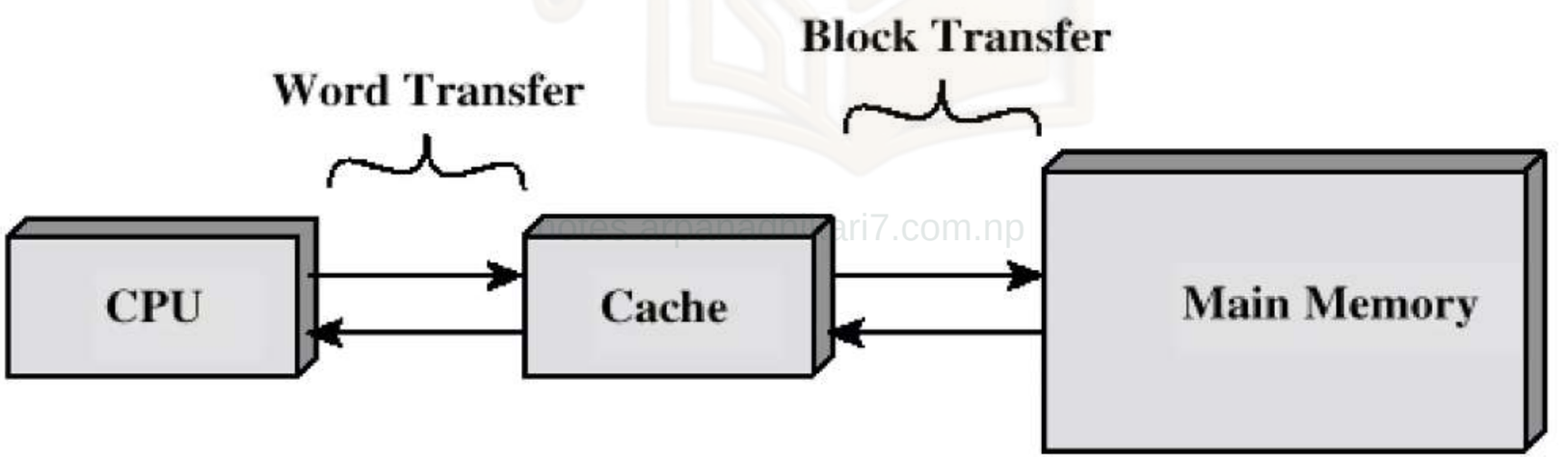
- Writing in an associative memory can take different forms. If entire memory is loaded with new information, then writing can be done by addressing each location in sequence.
- If unwanted words have to be deleted and new words inserted one at a time using tag register.
- A word is deleted for memory be clearing its tag bit to 0. After the new word is stored in memory it is made active by setting its tag bit to 1.

NANCY BEAU

Available on notes.arpanadhikari7.com.np

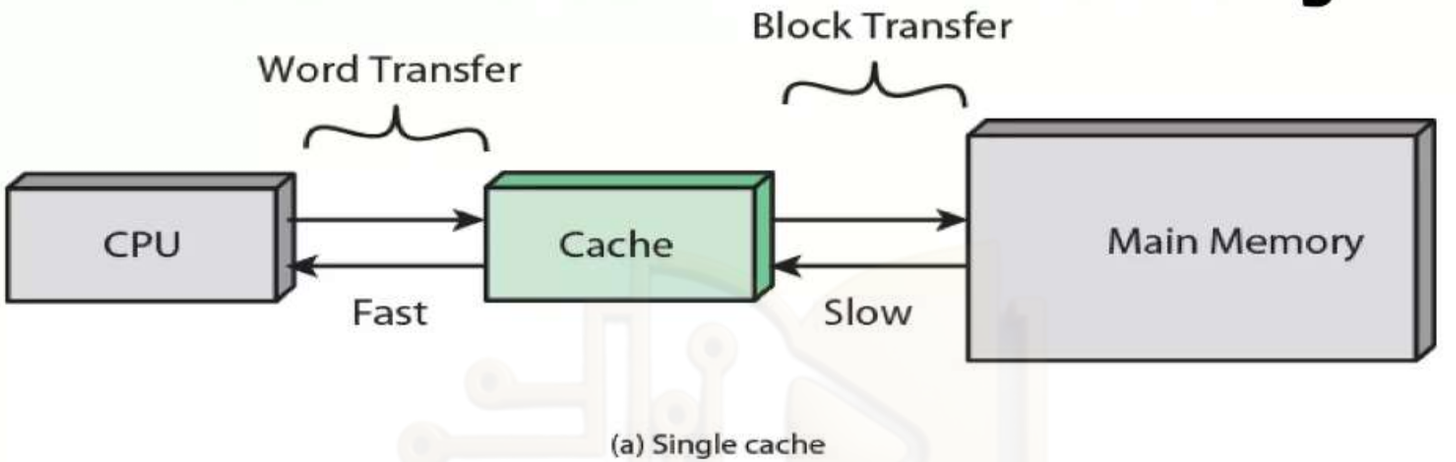
Cache

- Small amount of fast memory
- Sits between normal main memory and CPU
- May be located on CPU chip or module

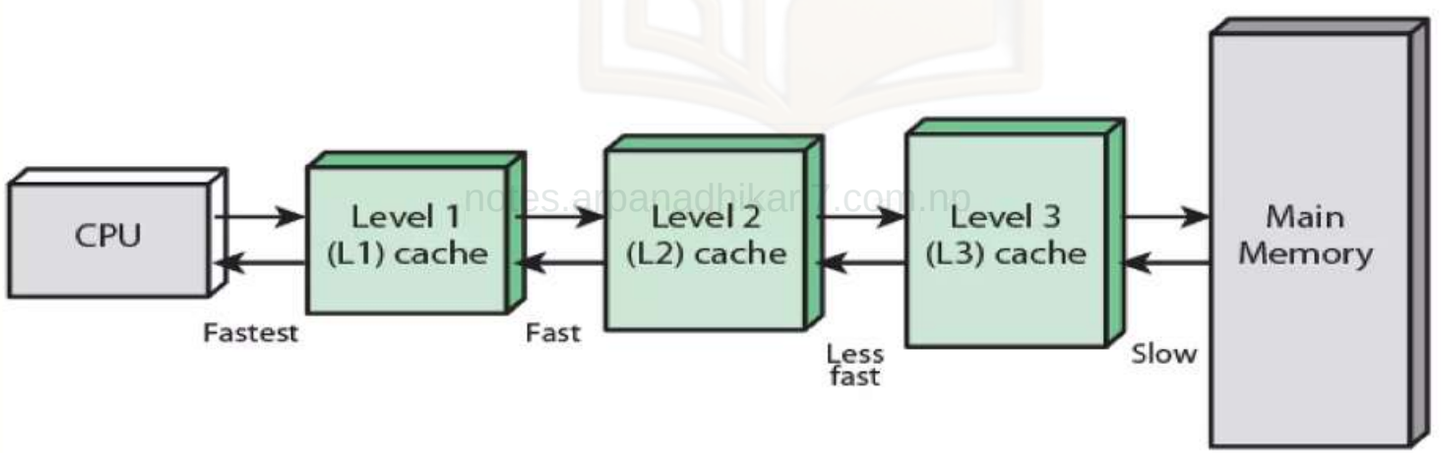


Available on notes.arpanadhikari7.com.np

Cache and Main Memory



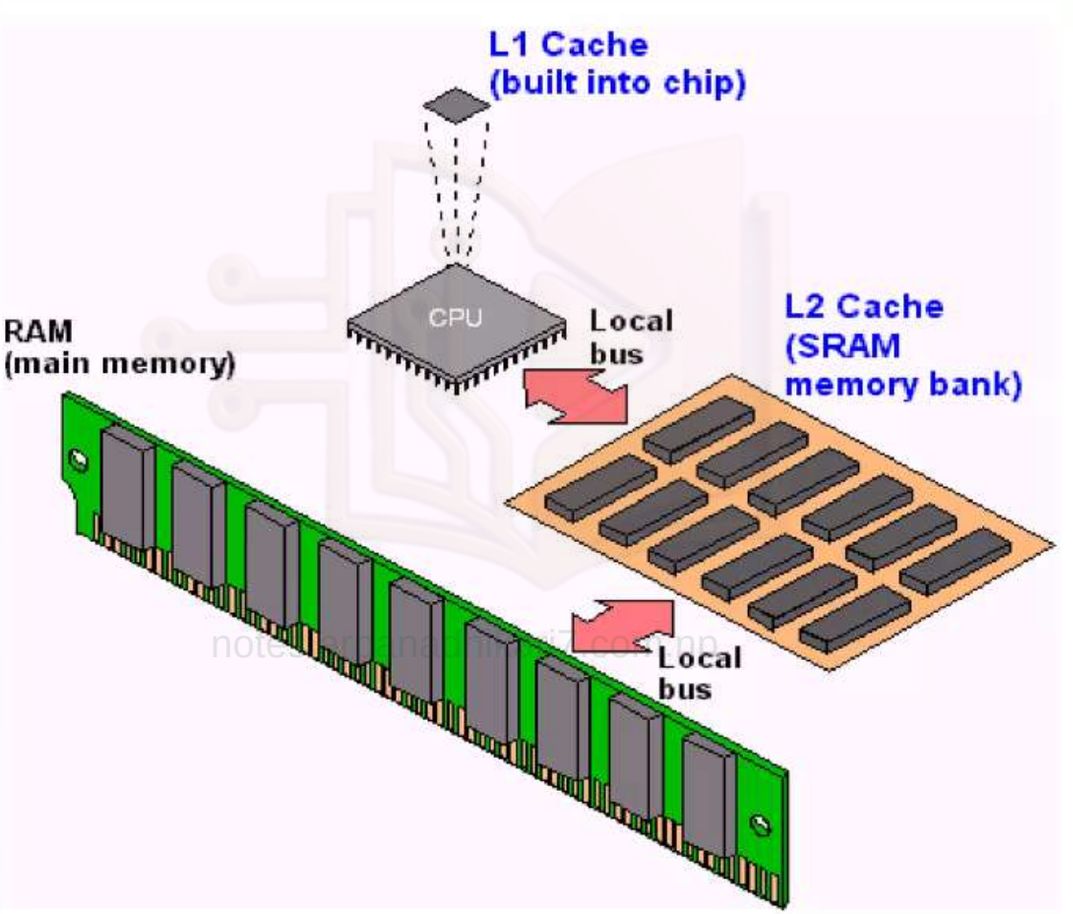
(a) Single cache



(b) Three-level cache organization

Available on notes.arpanadhikari7.com.np

Diagram showing different types of cache and their position in the computer system



Available on notes.arpanadhikari7.com.np

Cache memory

- Processor is much faster than the main memory.
  - As a result, the processor has to spend much of its time waiting while instructions and data are being fetched from the main memory.
  - Major obstacle towards achieving good performance.
- Speed of the main memory cannot be increased beyond a certain point.
- Cache memory is an architectural arrangement which makes the main memory appear faster to the processor than it really is.
- Cache memory is based on the property of computer programs known as "locality of reference".

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Locality of reference

- Cache memory is based on the concept of locality of reference.
  - If active segments of a program are placed in a fast cache memory, then the execution time can be reduced.
- Temporal locality of reference:
  - Whenever an instruction or data is needed for the first time, it should be brought into a cache. It will hopefully be used again repeatedly.
- Spatial locality of reference:
  - Instead of fetching just one item from the main memory to the cache at a time, several items that have addresses adjacent to the item being fetched may be useful.
  - The term "block" refers to a set of contiguous addresses locations of some size.

Available on notes.arpanadhikari7.com.np

Levels of Cache: Cache memory is categorized in levels based on it's closeness and accessibility to the microprocessor. There are three levels of a cache.

❖**Level 1(L1) Cache:** This cache is *inbuilt in the processor and is made of SRAM(Static RAM)* Each time the processor requests information from memory, the cache controller on the chip uses special circuitry to first check if the memory data is already in the cache. If it is present, then the system is spared from time consuming access to the main memory. In a typical CPU, primary cache ranges in size from 8 to 64 KB, with larger amounts on the newer processors. This type of Cache Memory is *very fast* because it *runs at the speed of the processor* since it is *integrated* into it.

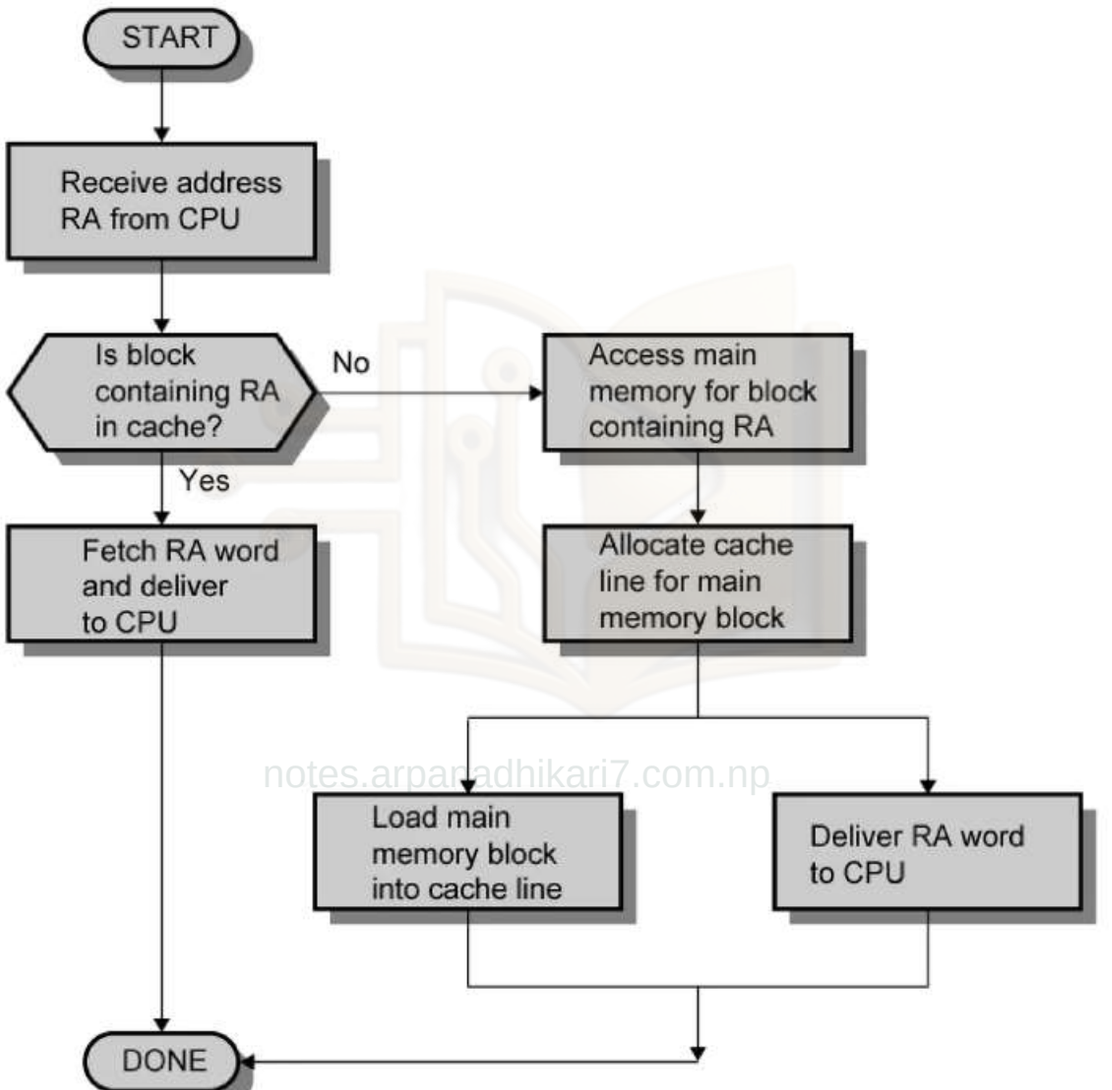
❖**Level 2(L2) Cache:** The L2 cache is larger but slower in speed than L1 cache. It is used to see recent accesses that is not picked by L1 cache and is usually 64 to 2 MB in size. A L2 cache is also found on the CPU. If L1 and L2 cache are used together, then the missing information that is not present in L1 cache can be retrieved quickly from the L2 cache. Like L1 caches, *L2 caches are composed of SRAM but they are much larger. L2 is usually a separate static RAM (SRAM) chip and it is placed between the CPU & DRAM(Main Memory)*

notes.arpanadhikari7.com.np

❖**Level 3(L3) Cache:** L3 Cache memory is an enhanced form of memory present on the motherboard of the computer. It is an extra cache built into the motherboard between the processor and main memory to speed up the processing operations. It reduces the time gap between request and retrieving of the data and instructions much more quickly than a main memory. L3 cache are being used with processors nowadays, having more than 3 MB of storage in it.

Available on notes.arpanadhikari7.com.np

Cache Read Operation - Flowchart



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Cache operation – overview

- CPU requests contents of memory location
- Check cache for this data
- If present, get from cache (fast)
- If not present, read required block from main memory to cache
- Then deliver from cache to CPU
- Cache includes tags to identify which block of main memory is in each cache slot

Available on notes.arpanadhikari7.com.np

Cache Design

- Size
- Mapping Function
- Replacement Algorithm
- Write Policy
- Block Size
- Number of Caches

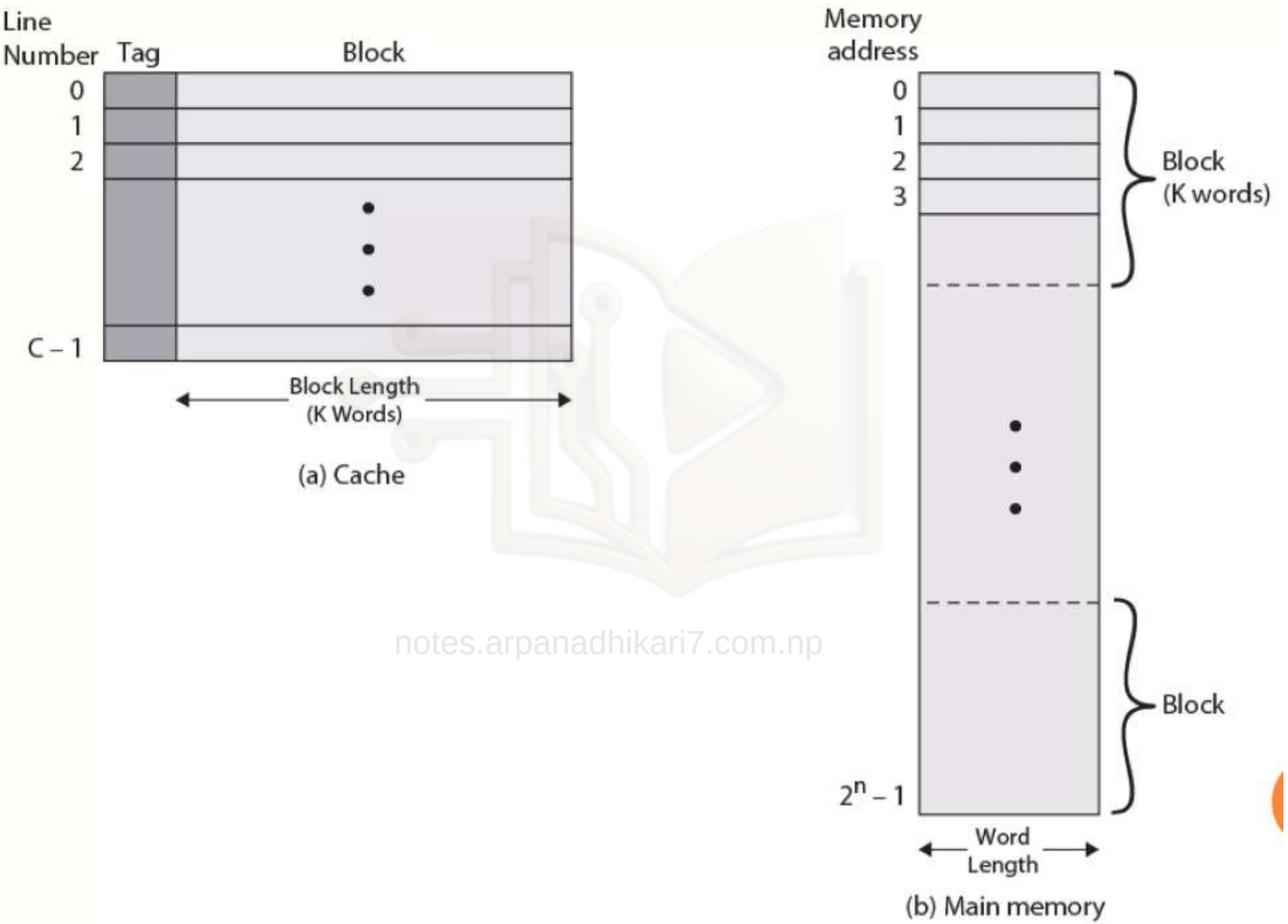
Size does matter

- Cost
  - More cache is expensive
- Speed
  - More cache is faster (up to a point)
  - Checking cache for data takes time

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Cache/Main Memory Structure



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Cache Memory Principles

- Block
  - The minimum unit of transfer between cache and main memory
- Frame
  - To distinguish between the data transferred and the chunk of physical memory, the term frame, or block frame, is sometimes used with reference to caches
- Line
  - A portion of cache memory capable of holding one block, so-called because it is usually drawn as a horizontal object
- Tag
  - A portion of a cache line that is used for addressing purposes
- Line size
  - The number of data bytes, or block size, contained in a line

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Cache Metrics /Cache Performance

Hit Ratio: $$HR = \frac{hits}{hits + misses} = 1 - MR$$

Miss Ratio: $$MR = \frac{misses}{hits + misses} = 1 - HR$$

Average Memory Access Time (AMAT):

$$AMAT = HitTime + MissRatio \times MissPenalty$$

- Goal of caching is to improve AMAT
- Formula can be applied recursively in multi-level hierarchies:

notes.arpanadhikari7.com.np

$$AMAT = HitTime_{L1} + MissRatio_{L1} \times AMAT_{L2} =$$

$$AMAT = HitTime_{L1} + MissRatio_{L1} \times (HitTime_{L2} + MissRatio_{L2} \times AMAT_{L3}) = \ldots$$

Available on notes.arpanadhikari7.com.np

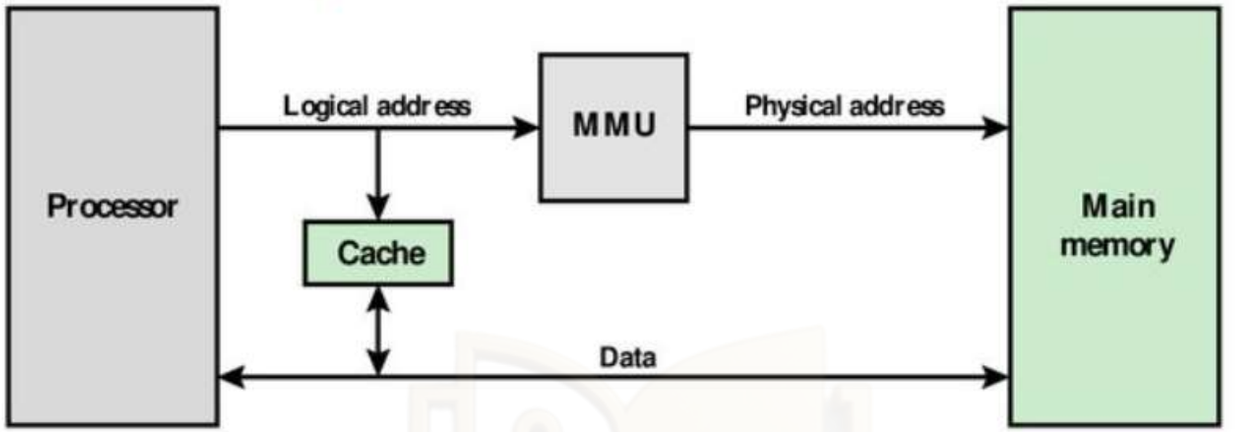
Cache Addresses

Virtual Memory

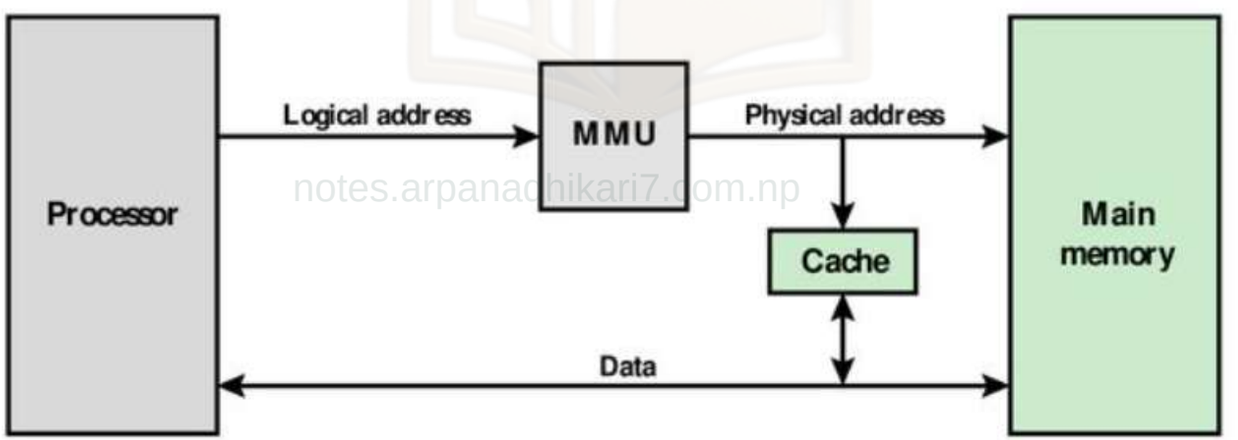
- Virtual memory
  - Facility that allows programs to address memory from a logical point of view, without regard to the amount of main memory physically available
  - When used, the address fields of machine instructions contain virtual addresses
  - For reads to and writes from main memory, a hardware memory management unit (MMU) translates each virtual address into a physical address in main memory

Available on notes.arpanadhikari7.com.np

Logical and Physical Caches



(a) Logical Cache



(b) Physical Cache

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Cache Mapping functions

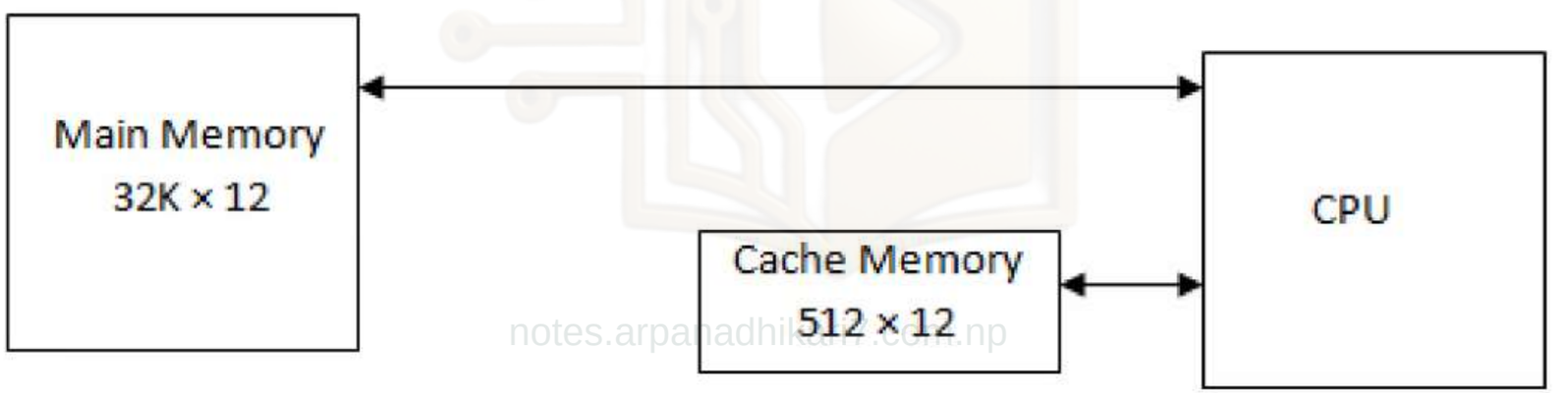
The address issued by the processor may correspond to that of an element that exists currently in the cache (cache hit); otherwise, it may correspond to an element that is currently residing in the main memory.

Therefore, address translation has to be made in order to determine the location of the requested element. This is one of the functions performed by the memory management unit (MMU).

Available on notes.arpanadhikari7.com.np

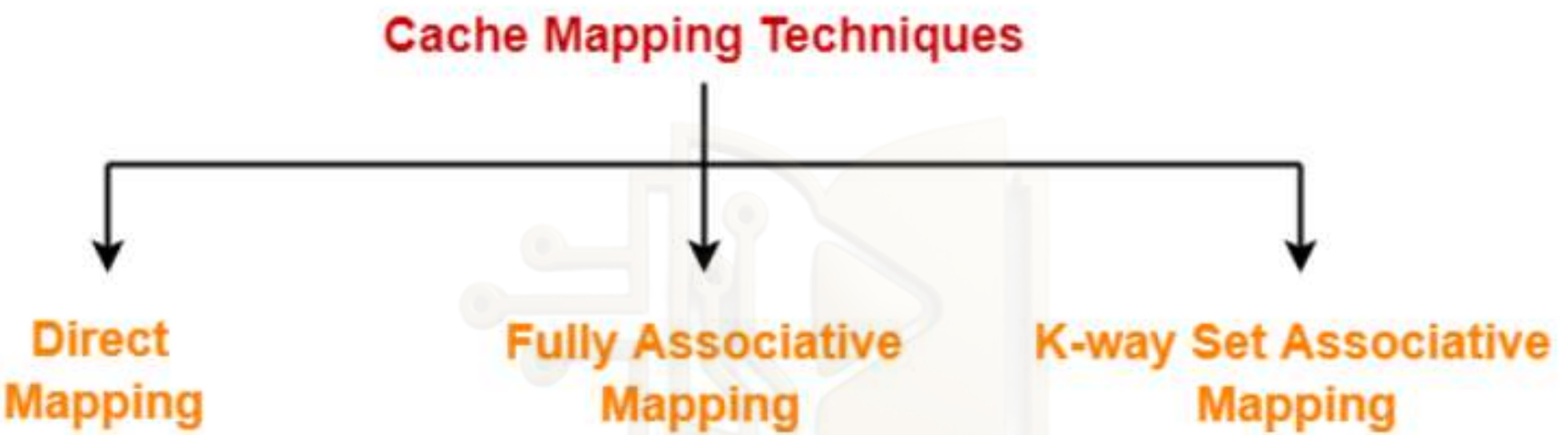
Example

A system with 512 x 12 cache and 32 K x 12 of main memory.



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

1. Direct Mapping

- The direct mapping technique is simple and inexpensive to implement.
- When the CPU wants to access data from memory, it places a address. The index field of CPU address is used to access address.
- The tag field of CPU address is compared with the associated tag in the word read from the cache.
- If the tag-bits of CPU address is matched with the tag-bits of cache, then there is a *hit* and the required data word is read from cache.
- If there is no match, then there is a *miss* and the required data word is stored in main memory. It is then transferred from main memory to cache memory with the new tag.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Cache Mapping Schemes

- Cache memory is smaller than the main memory
- Only few blocks can be loaded at the cache
- The cache does not use the same memory addresses
- **Which block in the cache is equivalent to which block in the memory?**
  - The processor uses Memory Management Unit (MMU) to convert the requested memory address to a cache address

Available on notes.arpanadhikari7.com.np

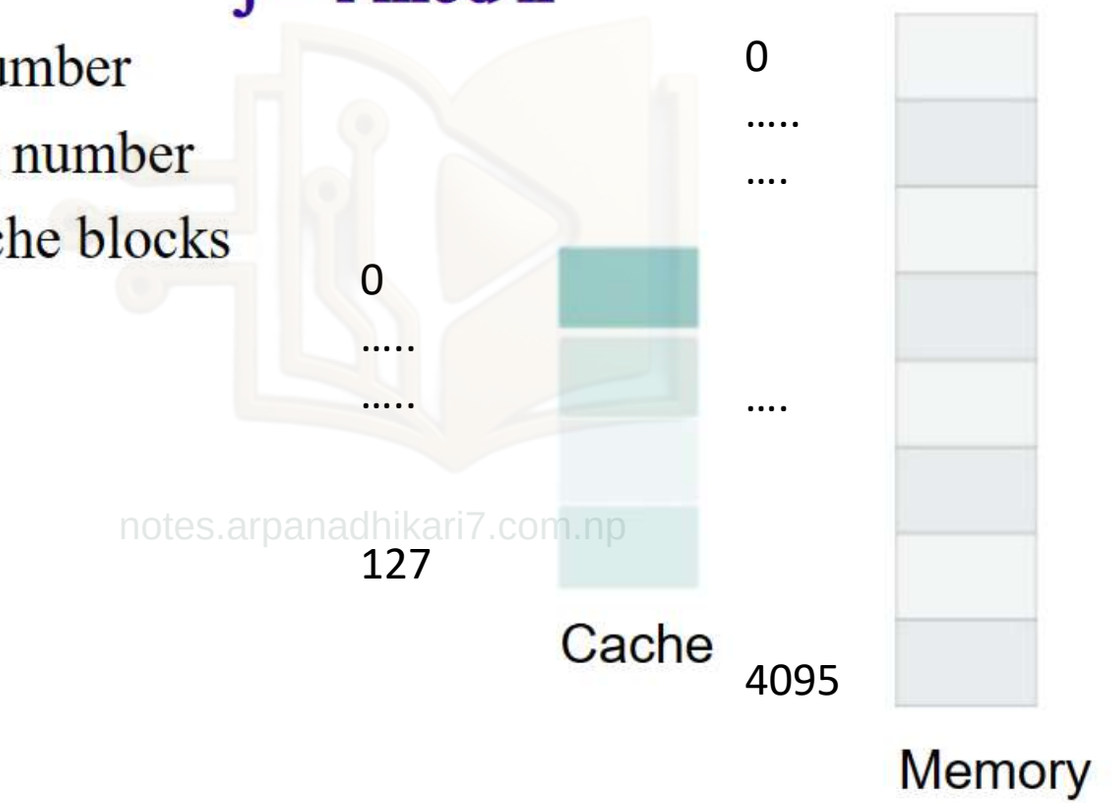
Direct Mapping

- **Assigns cache mappings using a modular approach**

$$j = i \bmod n$$

- j cache block number
- i memory block number
- n number of cache blocks

Let data bits of 16 bits.

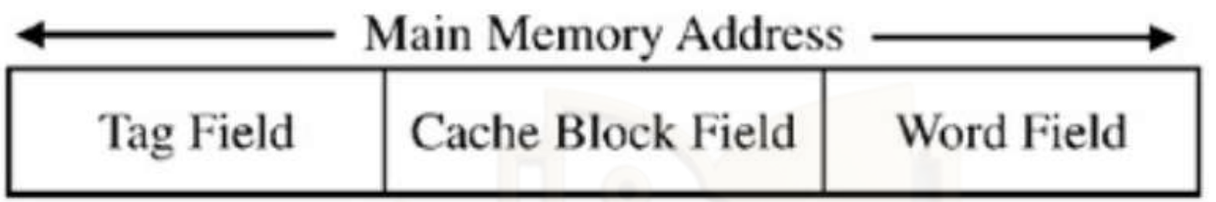


notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Direct Mapping (Cont.)

- Bits in the main memory address are divided into three fields.



- Word ➔ identifies specific word in the block

  Word field = $\log_2 B$, where B is the size of the block in words.

- Block ➔ identifies a unique block in the cache

  Block field = $\log_2 N$, where N is the size of the cache in blocks.

- Tag ➔ identifies which block from the main memory currently in the cache

  Tag field = $\log_2 (M/N)$, where M is the size of the main memory in blocks.

The number of bits in the main memory address = $\log_2 (B \times M)$

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Example (Cont.)

Word field $= \log_2 B = \log_2 16 = \log_2 2^4 = 4$ bits

Block field $= \log_2 N = \log_2 128 = \log_2 2^7 = 7$ bits

Tag field $= \log_2(M/N) = \log_2(2^2 \times 2^{10}/2^7) = 5$ bits

The number of bits in the main memory address $= \log_2 (B \times M) = \log_2 (2^4 \times 2^{12}) = 16$ bits.



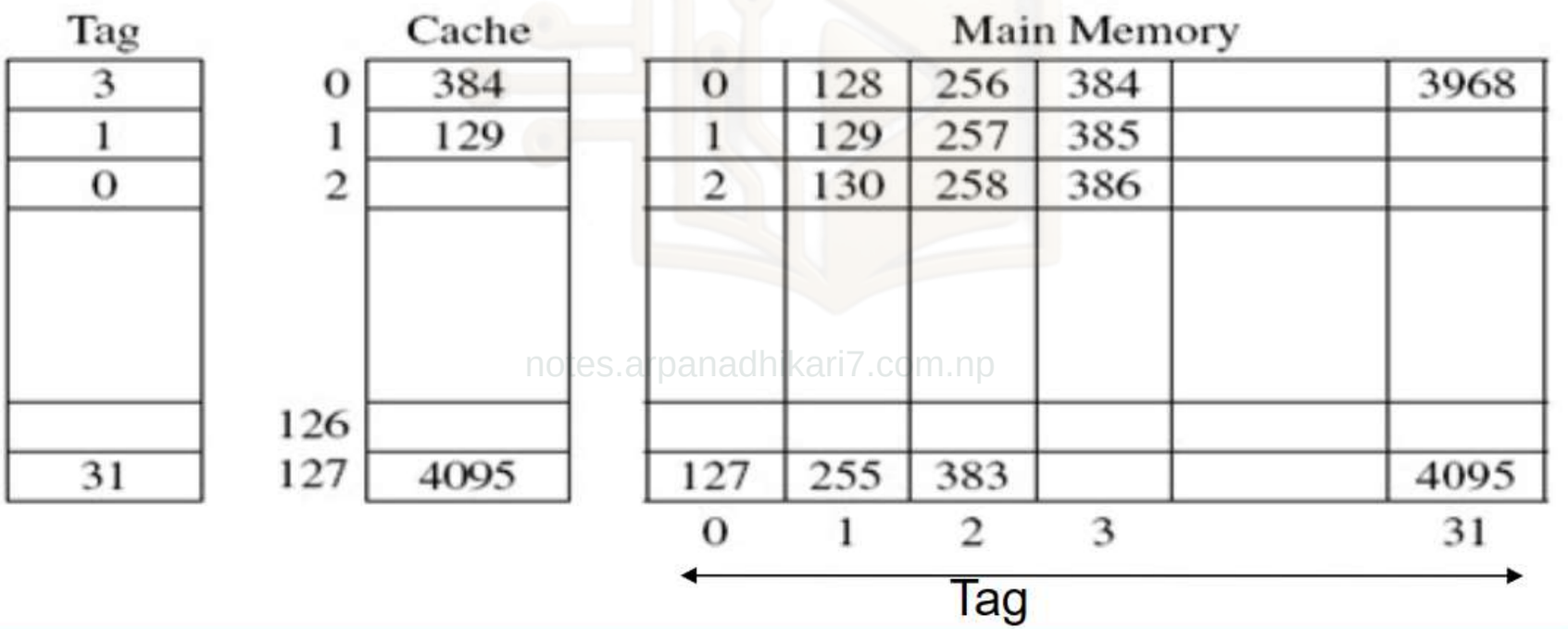
notes.arpanadhikari7.com.np

TAG LINE WORD

Available on notes.arpanadhikari7.com.np

Example

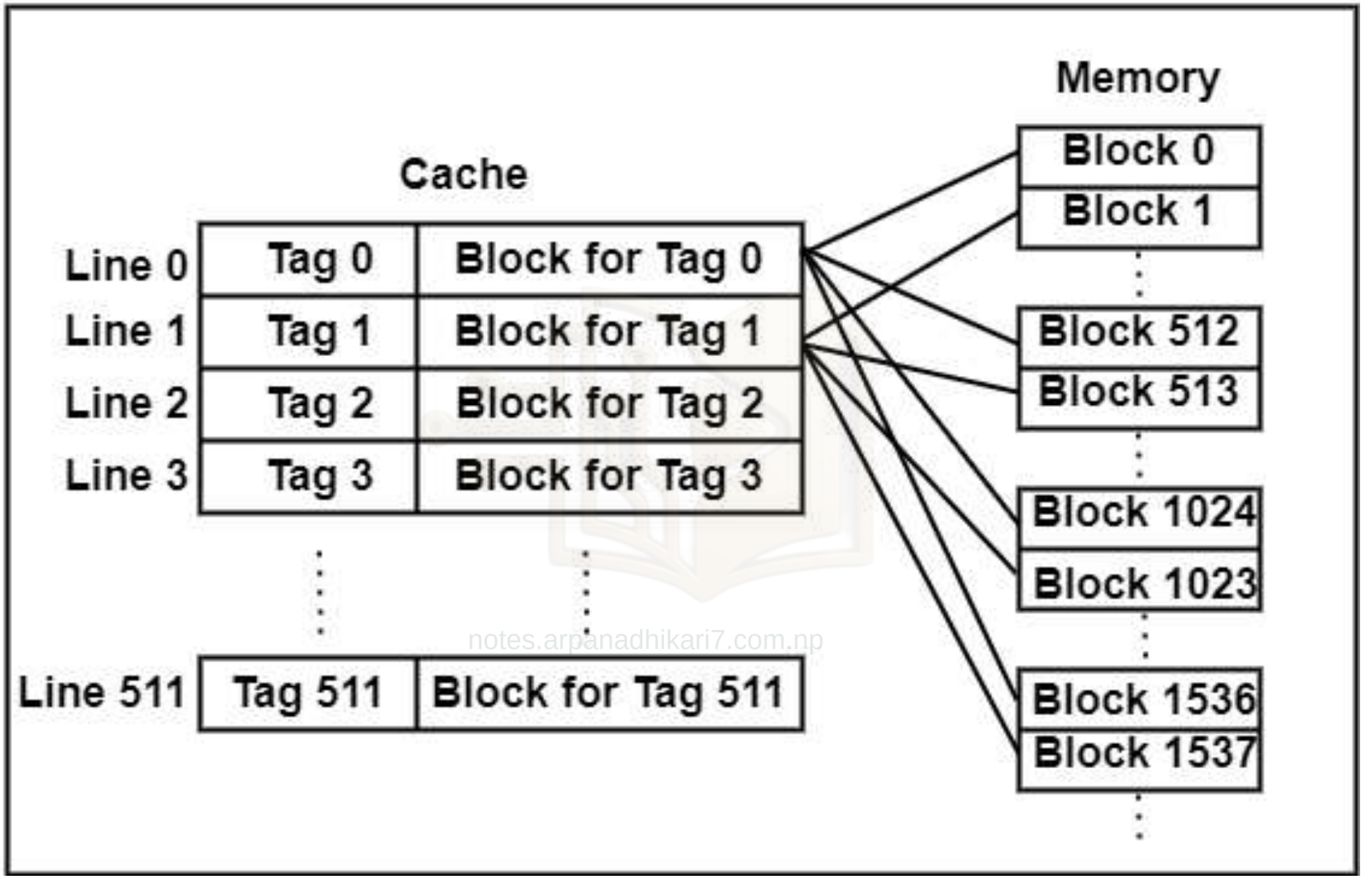
- Consider, for example, the case of a main memory consisting of 4K blocks, a cache memory consisting of 128 blocks, and a block size of 16 words. Show the direct mapping and the main memory address format?



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Direct Mapping of Main Memory to Cache



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

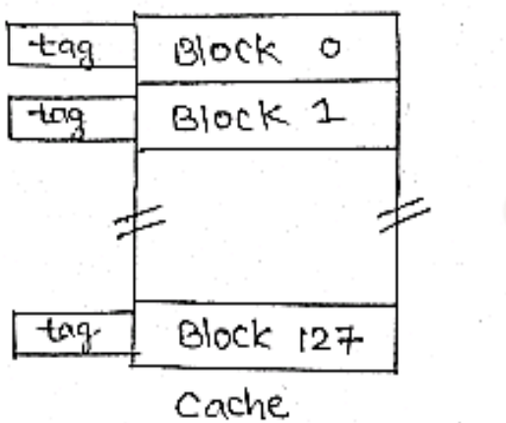
Direct Mapping

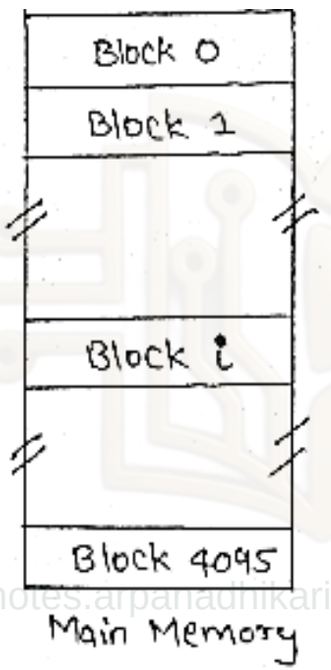
- Advantage
  - Easy
  - Does not require any search technique to find a block in cache
  - Replacement is a straight forward

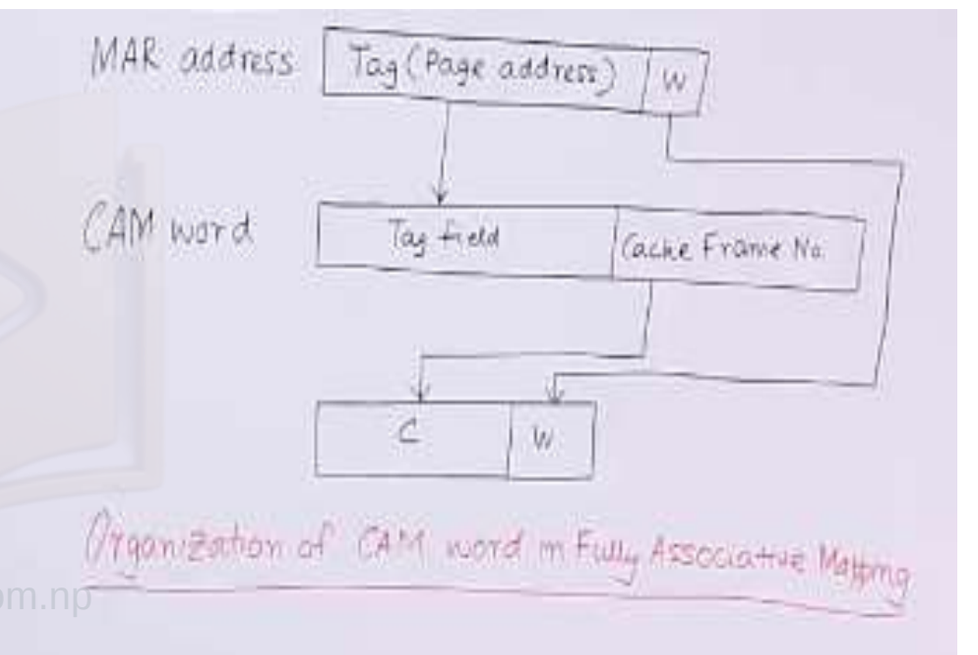
- Disadvantages
  - Many blocks in MM are mapped to the same cache block
  - We may have others empty in the cache
  - Poor cache utilization

Available on notes.arpanadhikari7.com.np

Associative mapping/ Fully associative mapping:







notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

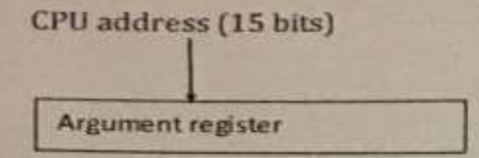
Contd…

- Flexible mapping method, in which main memory block can be placed into any cache block position.
- In this, 12 tag bits are required to identify a memory block when it is resident in the cache.
- CAM contains so many blocks with their respective cache location. It takes 12-bit tag(fig) from the memory address to find whether a particular block is in memory.
- The searching is done in all blocks within CAM and in parallel.
- Cost of associated mapped cache is higher than the cost of direct mapped because of the need to search all 128 tag pattern to determine whether a block is in cache. This is known as associative search.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

1. Associative mapping:



Address	Data
01000	3450
02777	6710
22345	1234

Figure: associative mapping

- The fastest and most flexible cache organization.
- Stores both address and data of memory word.
- The address value of 15 bits is shown as a five-digit octal number and its 12-bit word is shown as a four-digit octal number.
- A CPU address of 15 bits is placed in the argumented register and the associative memory is searched for matching address, if address is found the corresponding 12 bit is read and sent to CPU.
- If no match occurs the main memory is accessed for word also the address data pair must be placed in cache.
- If cache is full then what value to be displaced is determined by replacement algorithms

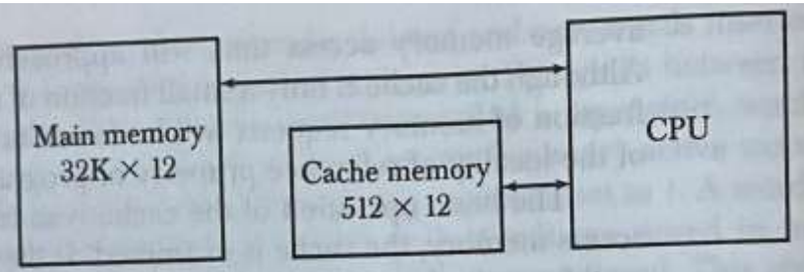


Figure 12-10 Example of cache memory.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Set-associative Mapping

In this mapping technique, blocks of cache are grouped to form a set and a block of main memory can go into any block of a specific set.

This form of mapping removes the drawbacks of direct mapping. In Set-associative mapping, each word that is present in the cache can have two or more words in the main memory for the same index address. Set associative cache mapping is a combination of direct and associative cache mapping techniques.
This also reduces searching overhead present in the associative mapping. Here, searching is restricted to the number of sets instead of the number of blocks.

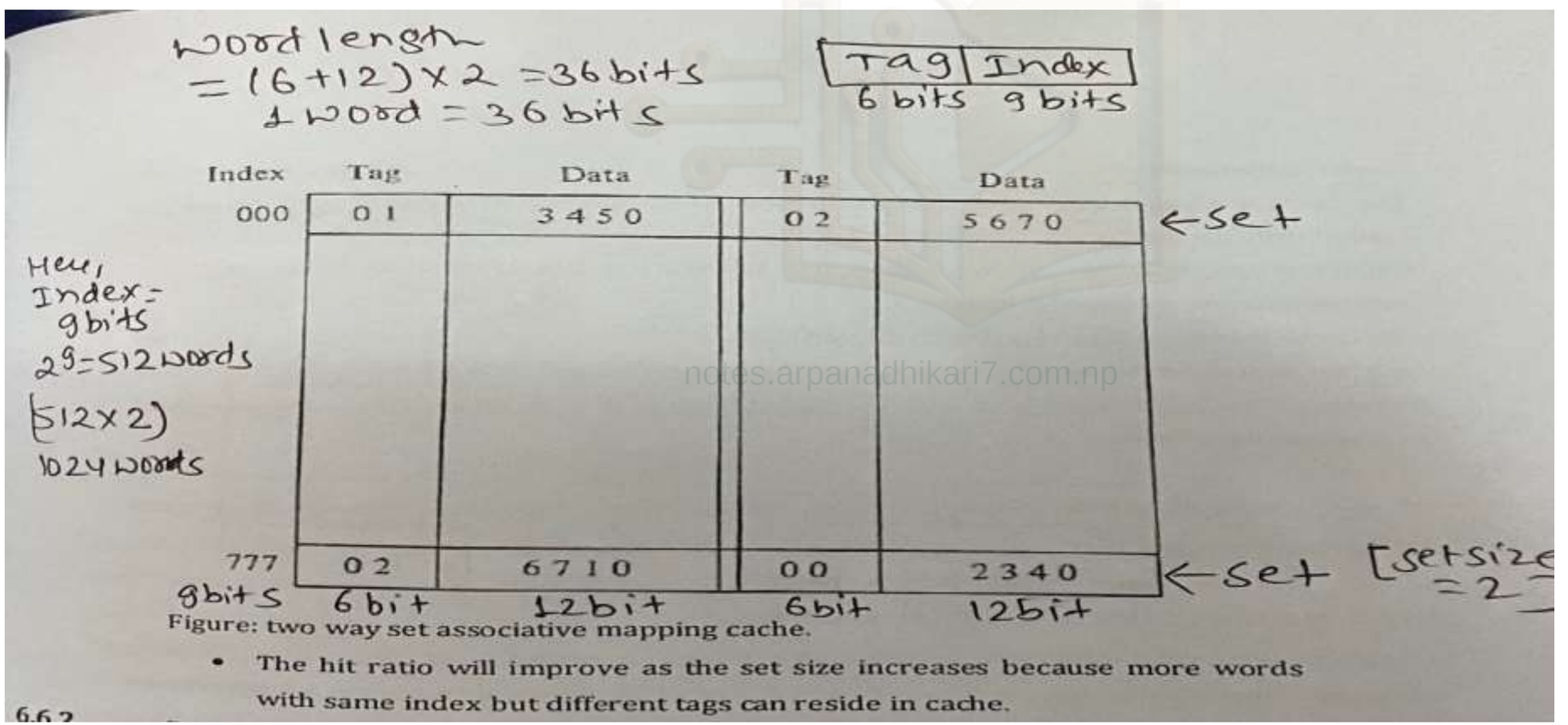


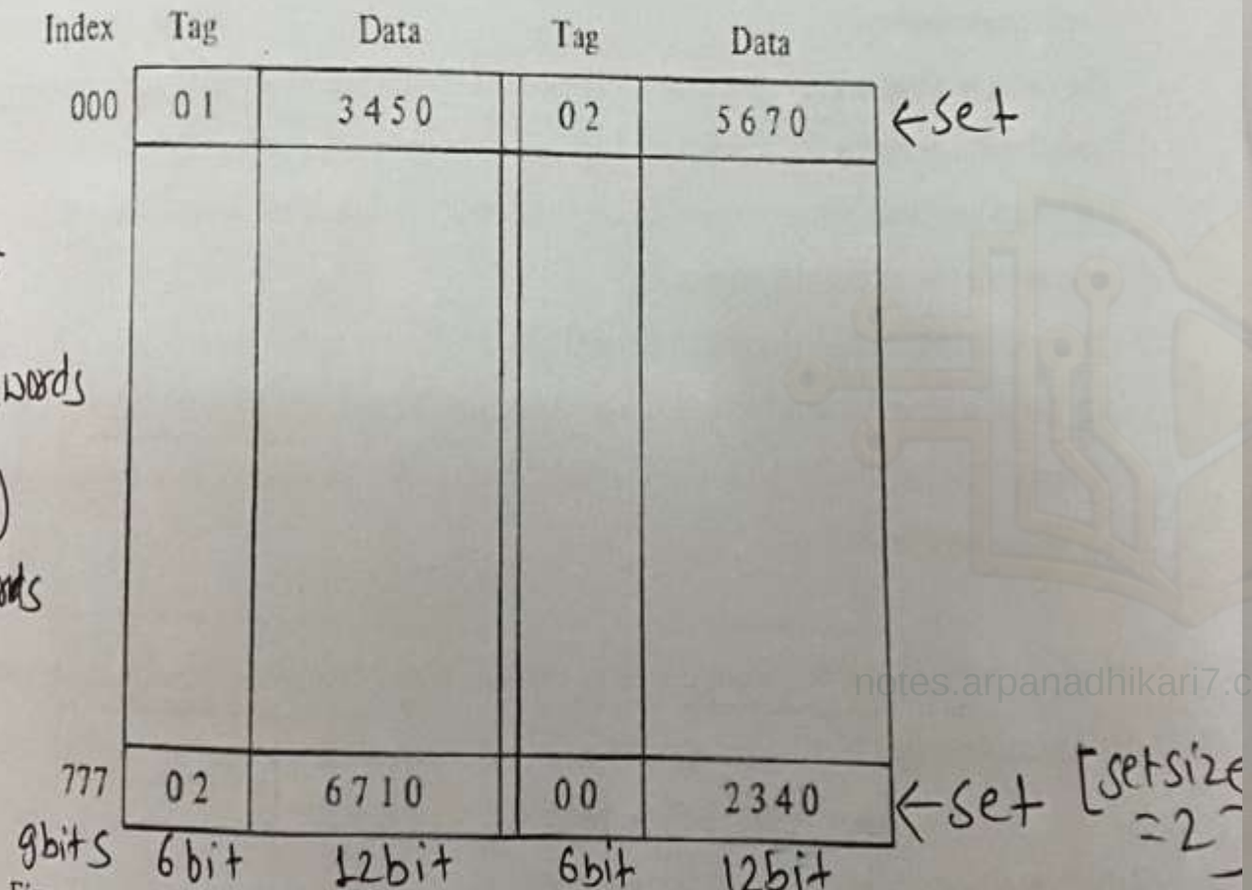
Figure: two way set associative mapping cache.

- The hit ratio will improve as the set size increases because more words with same index but different tags can reside in cache.

Available on notes.arpanadhikari7.com.np

word length
= (6+12) x 2 = 36 bits
1 word = 36 bits

Tag	Index
6 bits	9 bits



Hey, Index = 9 bits
2^9 = 512 words
(512 x 2)
1024 words

[setsize = 2

Figure: two way set associative mapping cache.

- The hit ratio will improve as the set size increases because more words with same index but different tags can reside in cache.

662

3. Set-Associate Mapping

- The disadvantage of direct mapping is it cannot store two words of same index with different tags.
- In set associate mapping two or more words of memory under same index address with different tags can be stored.
- Each data word is stored together with its tag and the number of tag-data item in one word of cache is said to form a set.
- For searching, CPU sends address first it searches for matched index in cache, after matching index it then matches the tag. For matched tag data is sent to CPU, if no match is found it is searched from main memory

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Writing policy with Caches

Write Back Cache

Cache policy where if a change is made to the page of cache storage then the corresponding page in main memory is not updated. Reads obviously do not need to be updated but writes do.

Write Through Cache

Cache policy where if a change is made to the page of cache storage then the corresponding page in main memory is also updated. Reads obviously do not need to be updated but writes do.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Data cache operates in two modes:

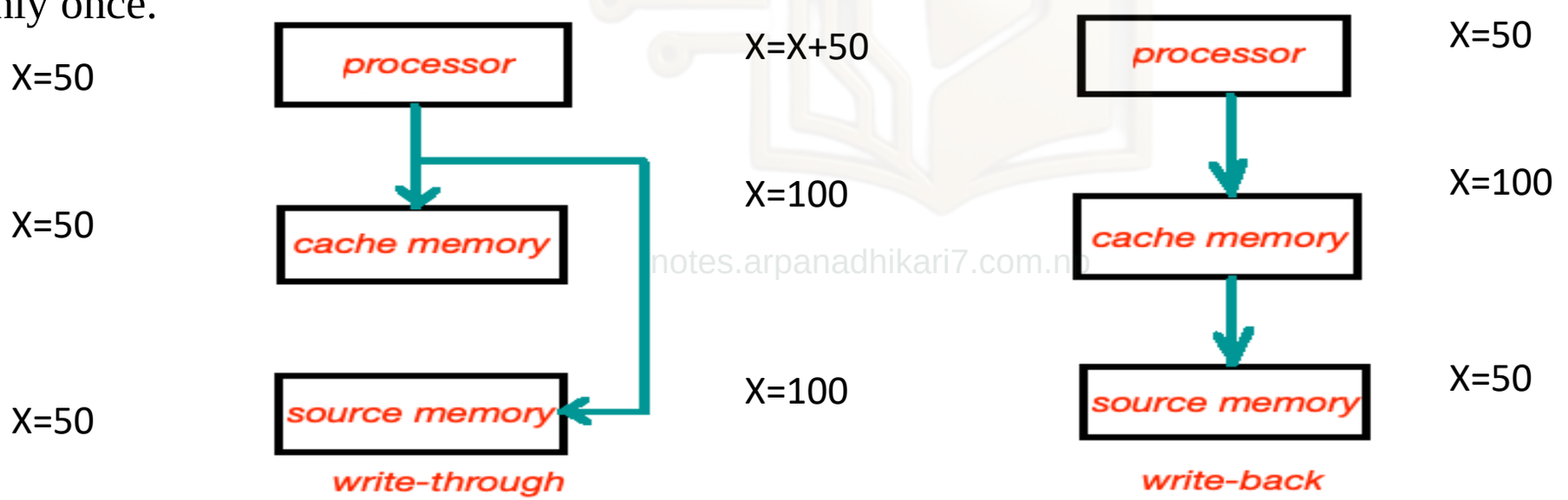
•write-through;

•write-back.

In case of write-through mode data is written in cache and in source memory when it is modified. No matter how many times data is modified in cache memory, it is the same amount of times modified in source memory as well. This mode can impact performance due to huge traffic through the bus between cache and source memory.

Write-back mode means that data is written only in cache. And when it is being replaced in cache memory it is written in back in source memory. Then source memory is always written only once.

Comparing write-through vs write-back data cache policy – write-back one is faster as memory source data is used only once.



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Page Replacement Algorithms

- Want lowest page-fault rate
- Evaluate algorithm by running it on a particular string of memory references (reference string) and computing the number of page faults on that string
- For example, a reference string is

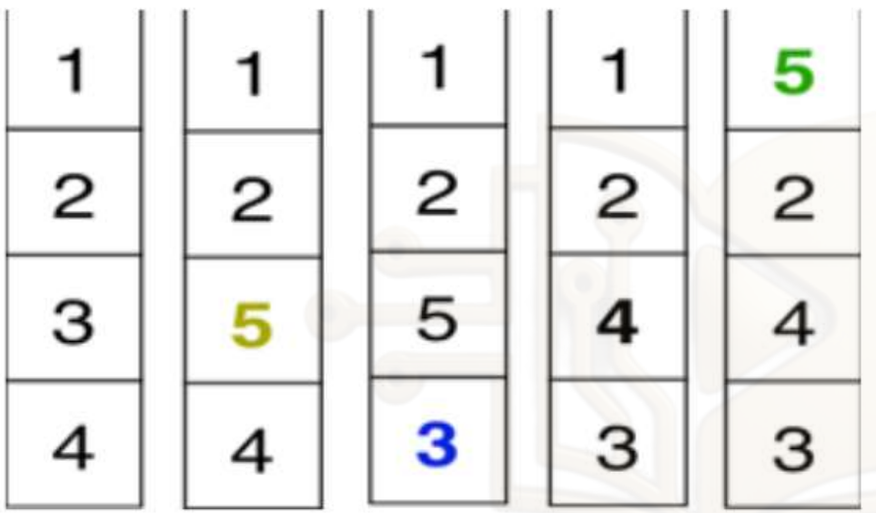
  1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

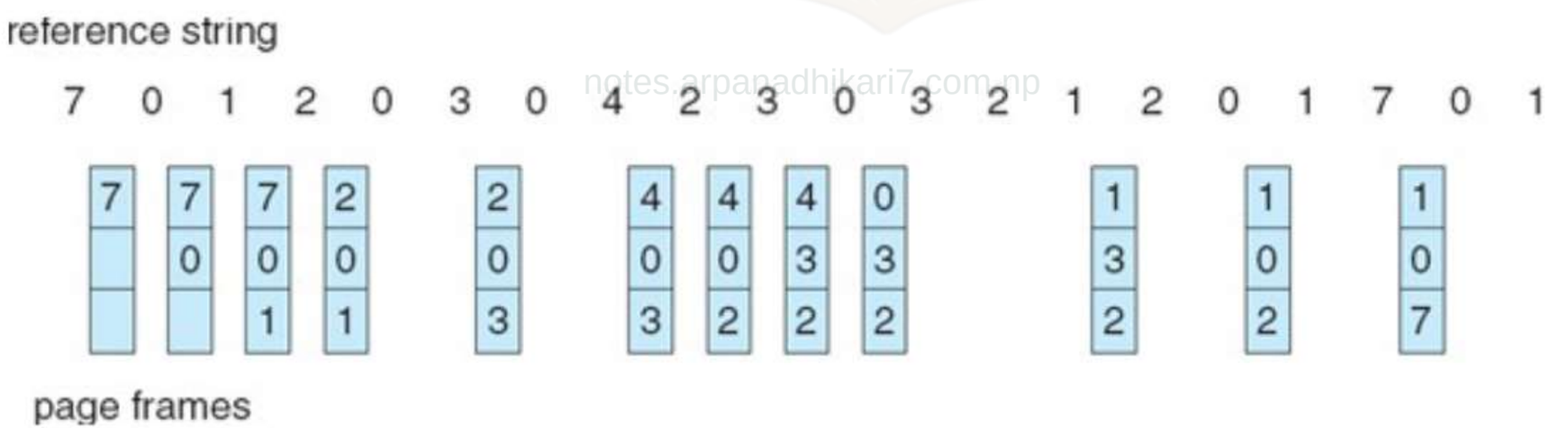
notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Least Recently Used (LRU) Algorithm

- Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5





notes.arpanadhikari7.com.np

LRU Page Replacement

- LRU is designed to take advantage of “normal” program behavior
- Programs are written to contain loops, which cause the main line of the code to execute repeatedly, with special-case code rarely being executed
- This set of pages that contain the code that is executed repeatedly is called the code locality of the process
- The LRU replacement algorithm is explicitly designed to take advantage of locality by assuming that if a page has been referenced recently, it is likely to be referenced again soon

Available on notes.arpanadhikari7.com.np

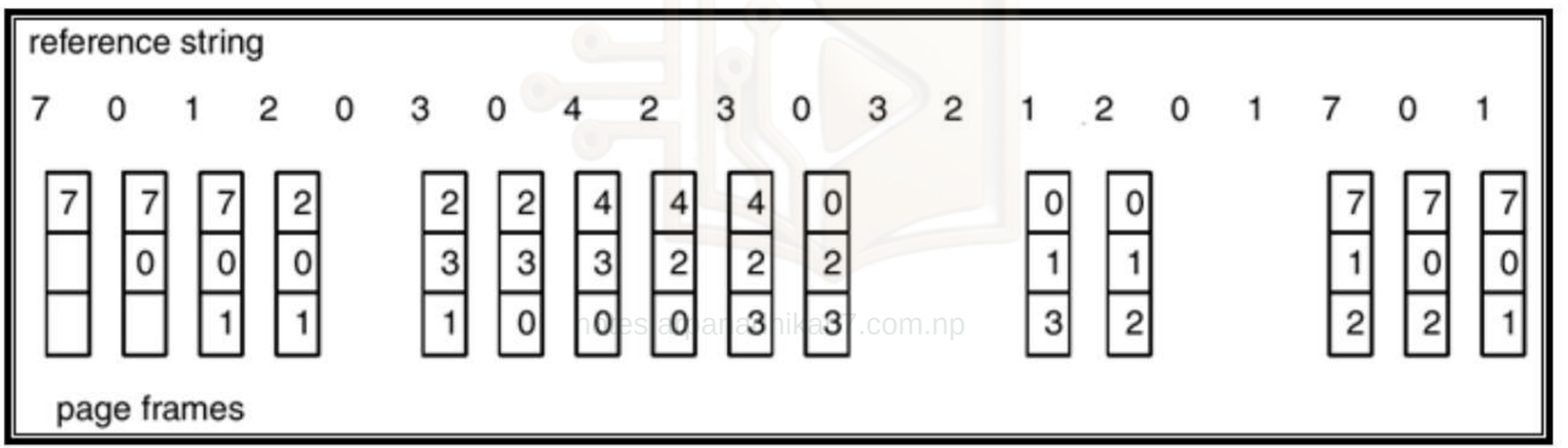
Least Recently Used (LRU) Algorithm

- LRU associates with each page the time of that page's last use
- When a page must be replaced, LRU chooses the page that has not been used for the longest period of time

- LRU is often used as a page replacement algorithm and is considered to be good
- The major problem is how to implement LRU replacement
- An LRU page replacement algorithm may require substantial hardware assistance

Available on notes.arpanadhikari7.com.np

FIFO Page Replacement



Here total page fault is 15.

Available on notes.arpanadhikari7.com.np

FIFO Page Replacement

- It replaces the page that has been in memory the longest
- Thus, it focuses on the length of time a page has been in memory rather than on how much the page is being used
- FIFO's main asset is that it is simple to implement
- Its behavior is not particularly well suited to the behavior of most programs, however, (it is completely independent of the locality of the program), so few systems use it

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Assignment

Explain in detail about Memory Management Hardware.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np